Visualisation I

Bar plots and distributions · 7AAVBCS5

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

11 November 2026



KING'S College LONDON

Lecture plan

How do chart choices change what we see in London venue records, borough summaries and bike hires?

- Why visualise? Exploration, explanation and misleading graphics
- Bar plots: categories, ordering, units and denominators
- Crosstabs: counts, composition and shared baselines
- Distributions: centre, spread, skew and unusual values
- Saving figures, live coding and applied exercises

Key learning outcomes

By the end of the week, you should be able to:

- choose between bar plots and histograms
- create readable bar plots from frequency tables and crosstabs
- use grouped, stacked and proportional bar plots appropriately
- describe distributions using mean, median, quartiles and range
- identify skew, outliers and suspicious values
- save figures in appropriate formats for reporting

The tables from Monday

Question	Table	Chart today
Entries of each type?	`value_counts()`	bar plot
Borough compositions?	`pd.crosstab()`	grouped, stacked or proportional bars
A numerical summary for each group?	`groupby().agg()`	bars comparing mean and median hire duration
Where do rates or durations cluster?	`describe()`	histogram

A chart inherits the table's **counting unit, classification, missingness and denominator**.

What we are plotting

File	Unit	Coverage used today
Venue extract	register entry	4,434 entries, 26 types; no missing borough/type labels
Borough context	administrative area	32 boroughs + City; 33 complete count, population, area and rate values
TfL hire extract	recorded hire	443,539 hires, 16–31 July 2025; 12 starting borough areas; complete duration and grouping fields

The **2023 release** combines layers captured largely in **2022**; population is **2015**. These rates compare mismatched dates, rather than measure current cultural access.

Why visualise?

1. **Choose the purpose:** explore data or explain a finding.
2. **Read examples:** see what a chart adds to a table or sentence.
3. **Check the design:** axes, positions, arithmetic and the story they suggest.
4. **Match the chart to the question:** categories, summaries or a distribution.

Explore and explain

Data → researcher → audience

Explore: understand the data	Explain: communicate a finding
Inspect distributions, gaps and unusual cases	Select evidence relevant to a question
Check assumptions and possible errors	Give readable labels, units and context
Develop hypotheses; return to the records	Make a claim the evidence supports

A rough histogram reveals the City’s unusual rate. A report figure must also explain its residential denominator.

Tables are sometimes better

Borough	Register entries	Entries per 10,000 residents
Westminster	653	28.32
City of London	111	137.04

Tables preserve exact values, combine labels and numbers, support lookup and can be sorted in different ways. A sentence may suffice for one comparison.

Use a chart when **position, shape or pattern** makes something easier to understand.

How many sevens?

1 7 1 8 9 0 4 6 2 8 4 3 8 5 1 8 6 2 7 5 4 2 3

9 3 1 8 7 3 2 8 9 3 4 9 3 5 8 8 1 4 3 7 2 2 3

9 8 2 1 4 3 9 8 7 4 5 2 1 9 6 3 8 5 4 2 7 9 5

4 5 2 3 6 5 1 5 8 2 6 3 2 5 8 4 1 2 3 8 1 3 2

4 1 8 2 2 1 4 5 6 3 1 4 7 3 3 2 5 7 1 1 9 3 4

2 3 6 8 5 0 1 2 5 4 8 9 6 3 1 5 4 1 2 6 6 8 2

8 1 3 5 8 5 2 5 8 4 6 3 2 9 7 1 2 4 7 9 2 6 3

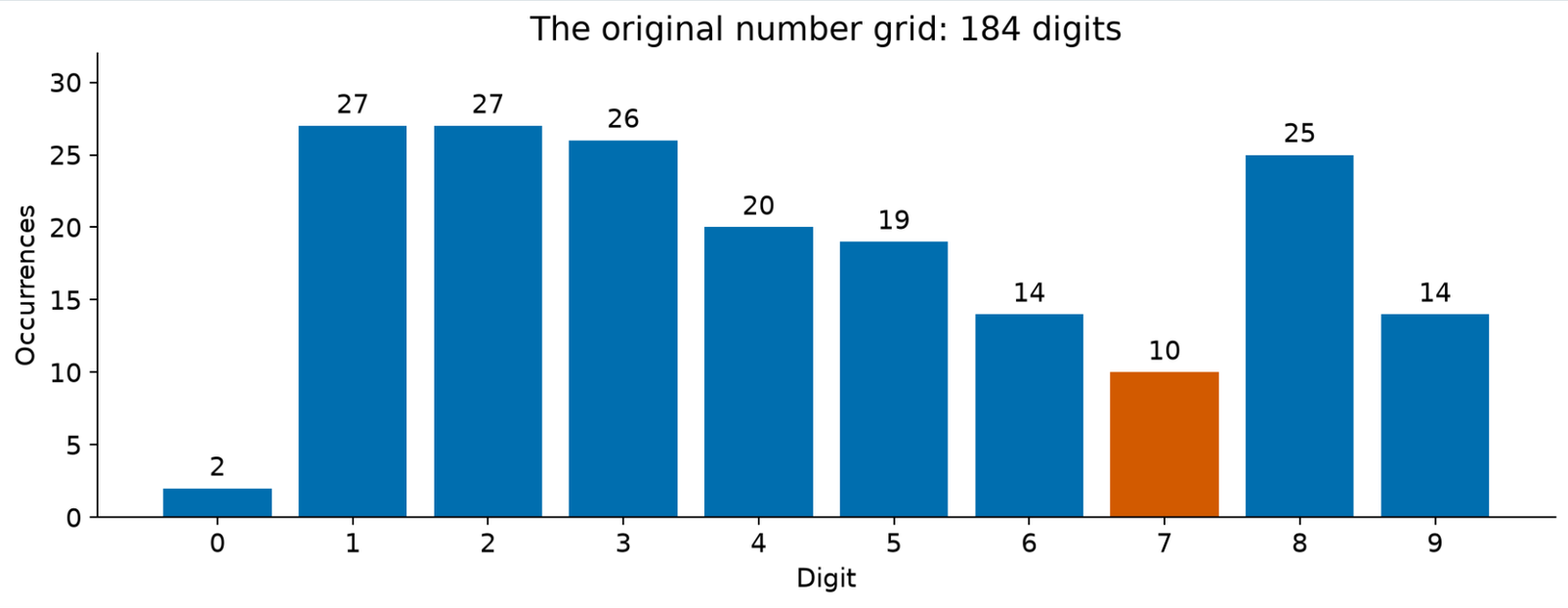
6 1 5 1 3 3 2 1 8 2 6 8 4 1 9 3 1 8 4 1 1 8 9

The same numbers, with sevens highlighted

1	**7**	1	8	9	0	4	6	2	8	4	3	8	5	1	8	6	2	**7**	5	4	2	3
9	3	1	8	**7**	3	2	8	9	3	4	9	3	5	8	8	1	4	3	**7**	2	2	3
9	8	2	1	4	3	9	8	**7**	4	5	2	1	9	6	3	8	5	4	2	**7**	9	5
4	5	2	3	6	5	1	5	8	2	6	3	2	5	8	4	1	2	3	8	1	3	2
4	1	8	2	2	1	4	5	6	3	1	4	**7**	3	3	2	5	**7**	1	1	9	3	4
2	3	6	8	5	0	1	2	5	4	8	9	6	3	1	5	4	1	2	6	6	8	2
8	1	3	5	8	5	2	5	8	4	6	3	2	9	**7**	1	2	4	**7**	9	2	6	3
6	1	5	1	3	3	2	1	8	2	6	8	4	1	9	3	1	8	4	1	1	8	9

The distribution reveals the gap

Digit	0	1	2	3	4	5	6	7	8	9
Count	2	27	27	26	20	19	14	10	25	14



Seven occurs **10 times among 184 digits**. Its low frequency is easier to see against the other counts than in the grid.

Why visualisation matters

- **Discovery:** a distribution exposes a gap or tail a mean hides.
- **Quality control:** an isolated value tells us where to investigate.
- **Communication:** a comparison becomes visible before every number is read.
- **Argument:** design directs attention towards an interpretation.

> “The simple graph has brought more information to the data analyst’s mind than any other device.”

John W. Tukey, *The Future of Data Analysis* (1962)

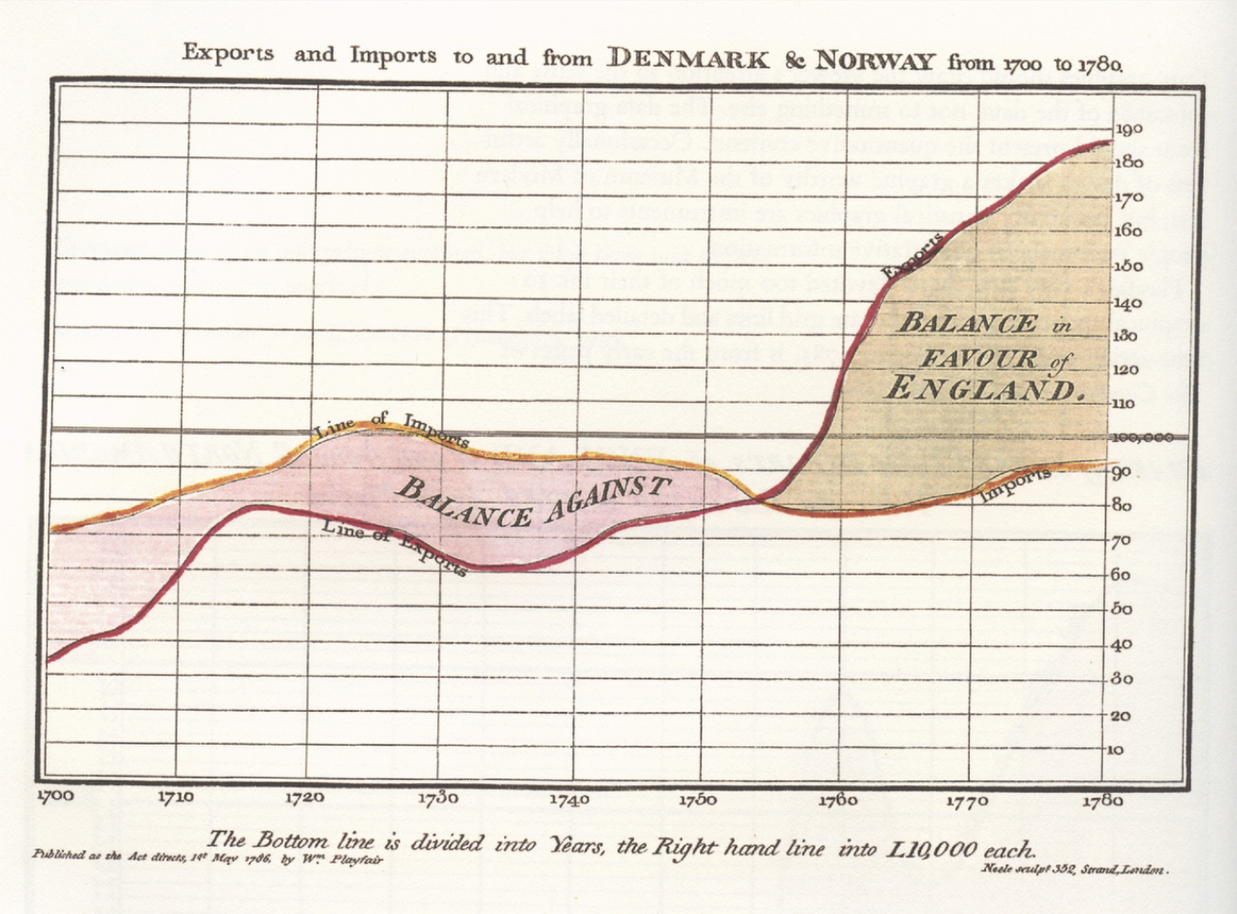
What a chart should add

Visualisation has a long history across statistics, design, cartography and the humanities.

A useful figure makes a pattern or comparison easier to see than the raw table or a short sentence does.

For London, 33 exact rates belong in a table for lookup. Their concentration near the low end, and one long right tail, are easier to grasp in a histogram.

Playfair: trade as a visual argument



William Playfair, *Commercial and Political Atlas* (1786).

- Years: 1700–1780
- Trade value in pounds; grid divisions £10,000
- Shading: balance “against” or “in favour of England”

The chart presents a political-economic interpretation, not just two lines.

Snow: a persuasive map, a qualified story



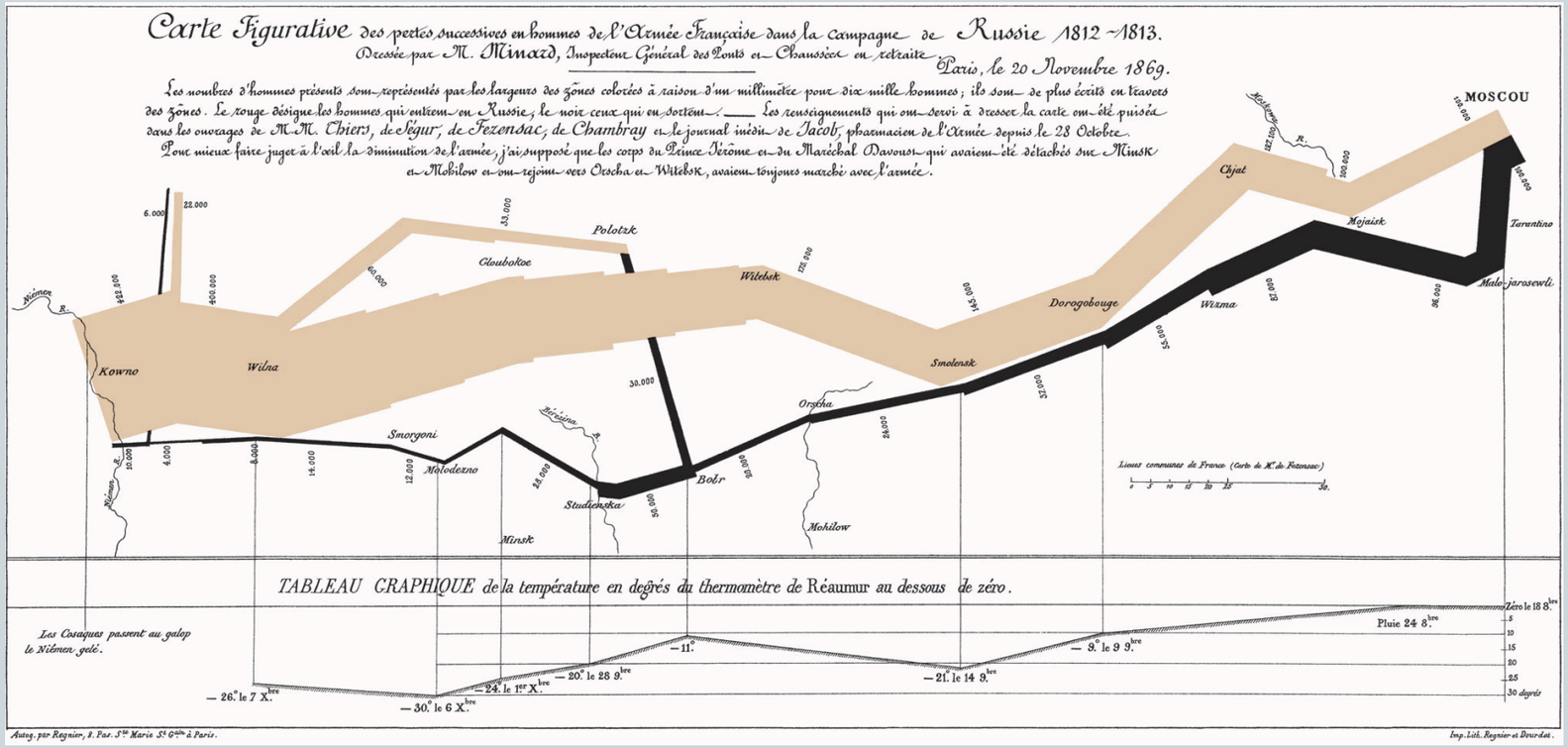
Each short black bar marks one cholera death at a recorded address. Stacks of bars show multiple deaths; the pumps are labelled separately.

Soho, 1854: deaths cluster around the Broad Street pump, on today's Broadwick Street.

Snow combined mapped deaths with inquiries about water use. Authorities removed the handle on **8 September**.

The outbreak was already declining. The later published map supports an argument; it does not prove that removing the handle ended the outbreak.

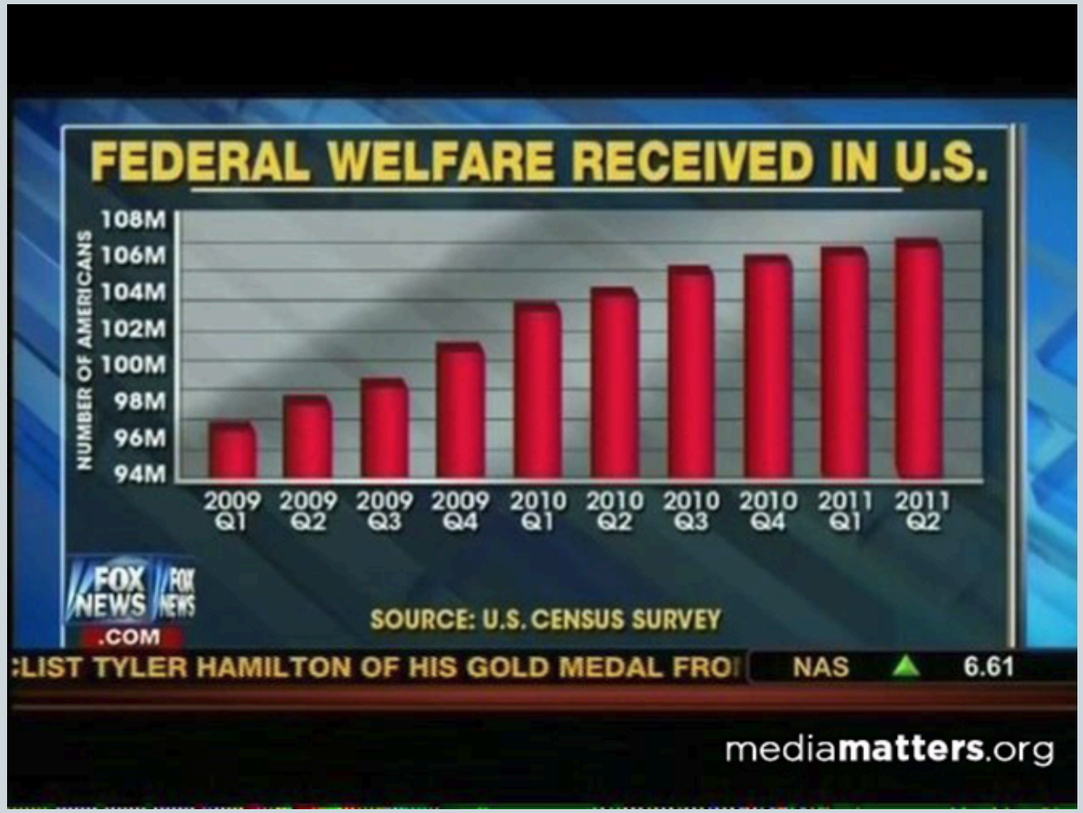
Minard: several variables in one image



Width: **troop numbers**. Position: **location**. Colour: **outward/return journey**. Dates and temperature accompany the retreat.

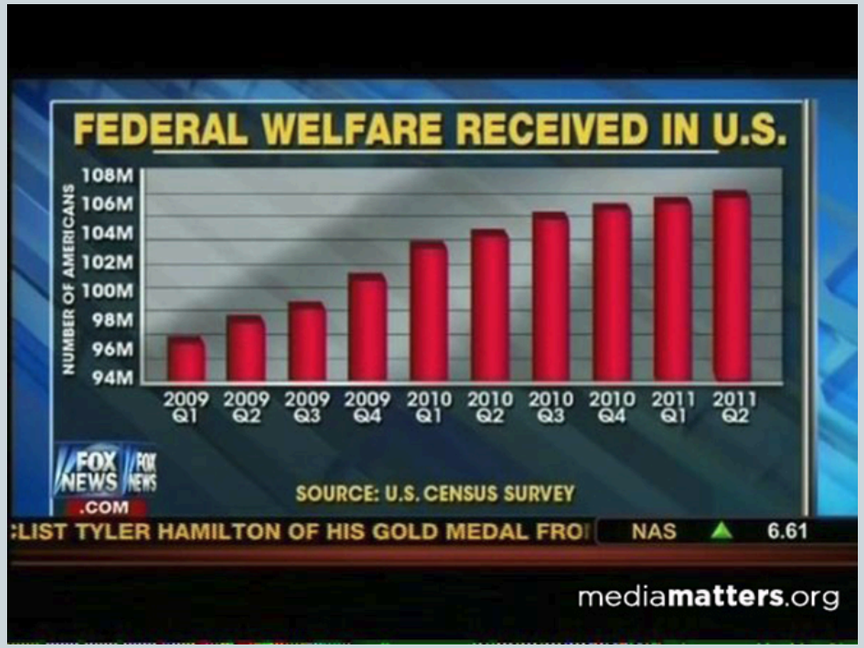
Several variables tell one story, but the reader must learn the encodings. The temperature scale is **Réaumur**, not Celsius.

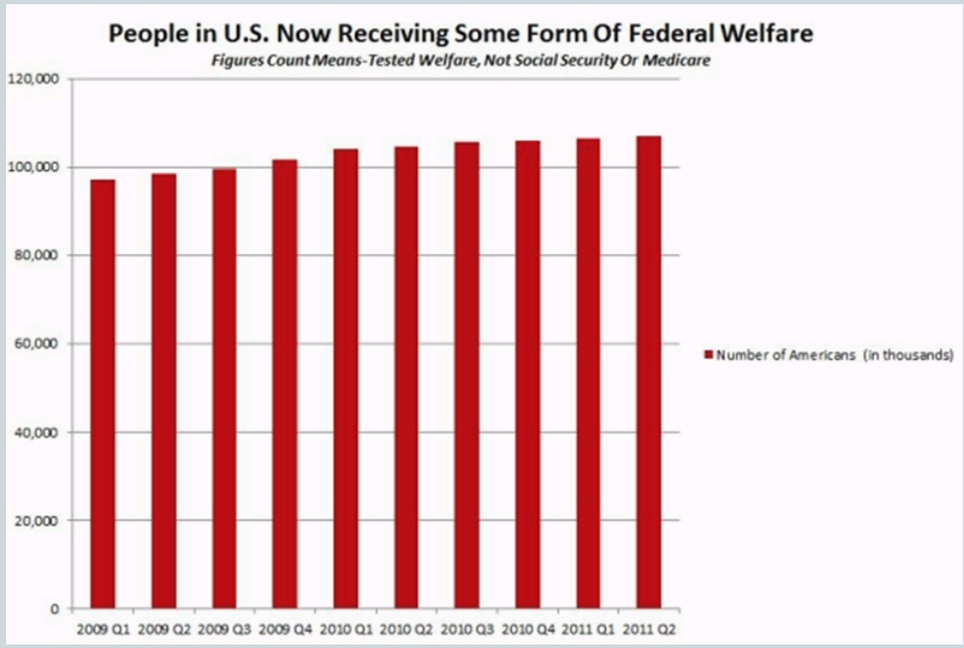
Read the broadcast chart



The vertical axis starts at **94 million**, not zero. How large does the increase look before you read that scale?

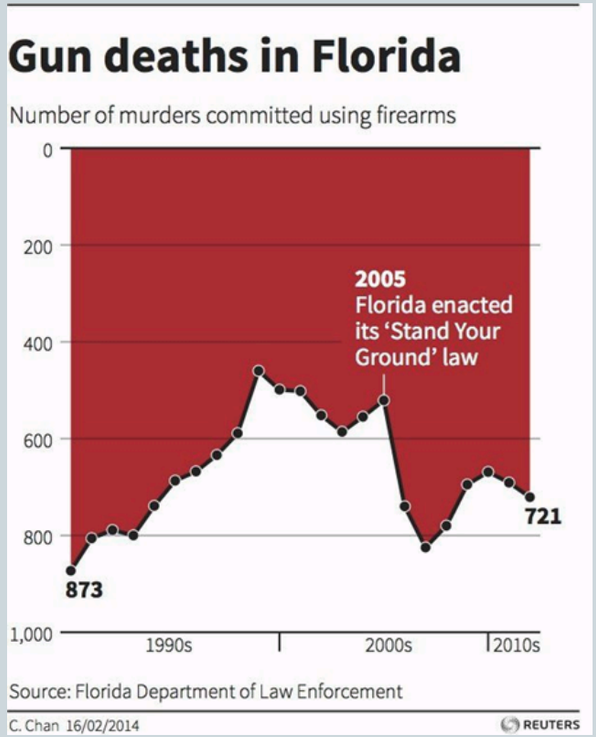
A zero baseline changes the impression





Same displayed series, different visual impression. Bars encode magnitude by length from a baseline. Removing the lower 94 million exaggerates relative change. Use zero for count bars.

Axis direction changes a story

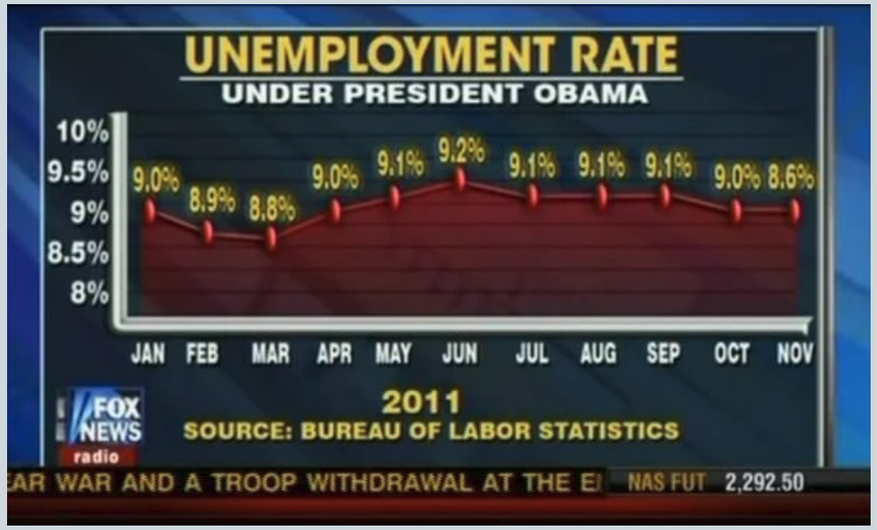


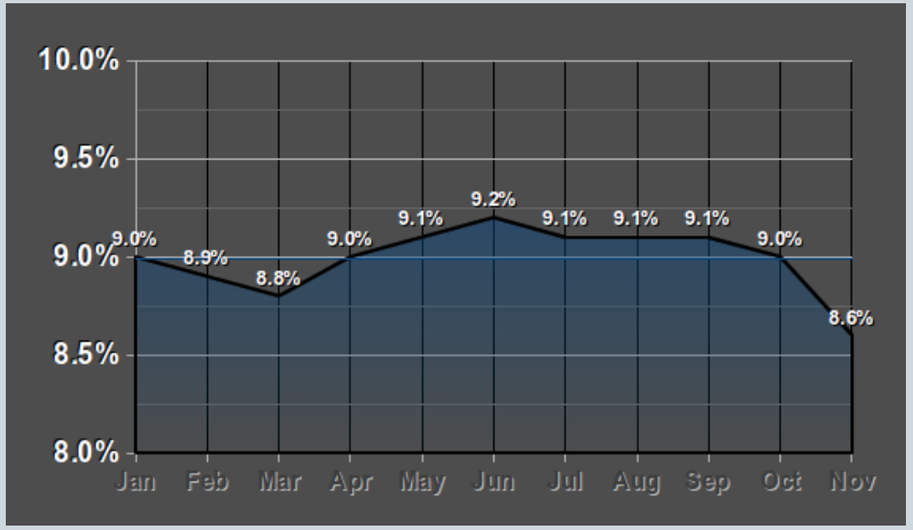
The Florida graphic places **zero at the top**. Downward movement represents a higher count.

Axis direction, scale and event annotations guide a narrative before the reader checks values.

A policy-date annotation does not establish causation. Visualisation II develops that explanation problem.

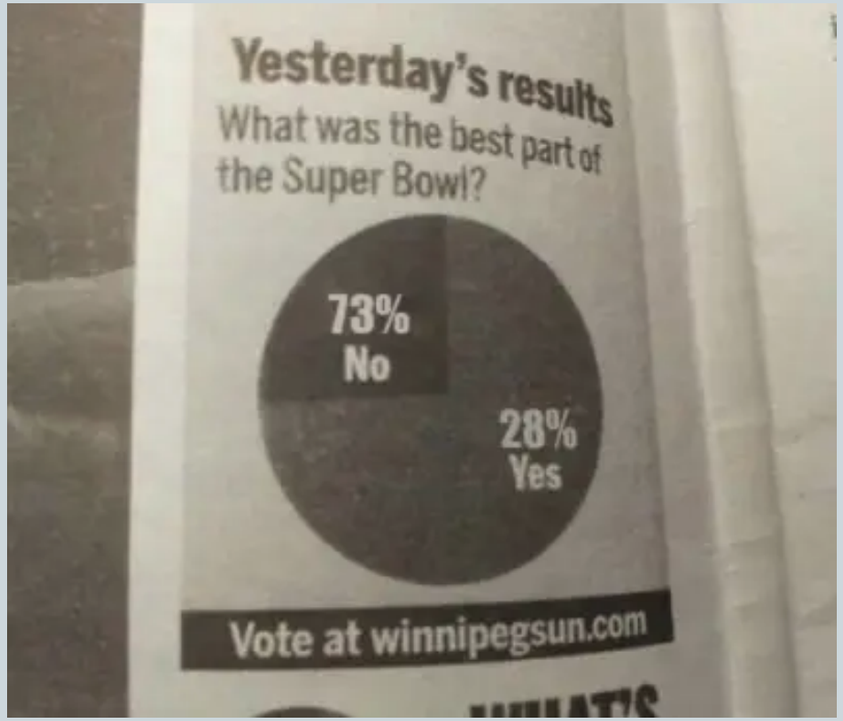
Position should agree with the value





On an upward-increasing axis, the final **8.6%** point should be below earlier **8.8%** and **8.9%** points. Check labels against positions, not just apparent line direction.

Check arithmetic as well as the picture



The Sun poll labels **73% No** and **28% Yes**: a total of **101%**.

Rounding can sometimes produce 101%, but the sector sizes also fail to match the labels.

Check totals, denominators and geometry. A humorous error is still a useful audit prompt.

Choosing the right chart

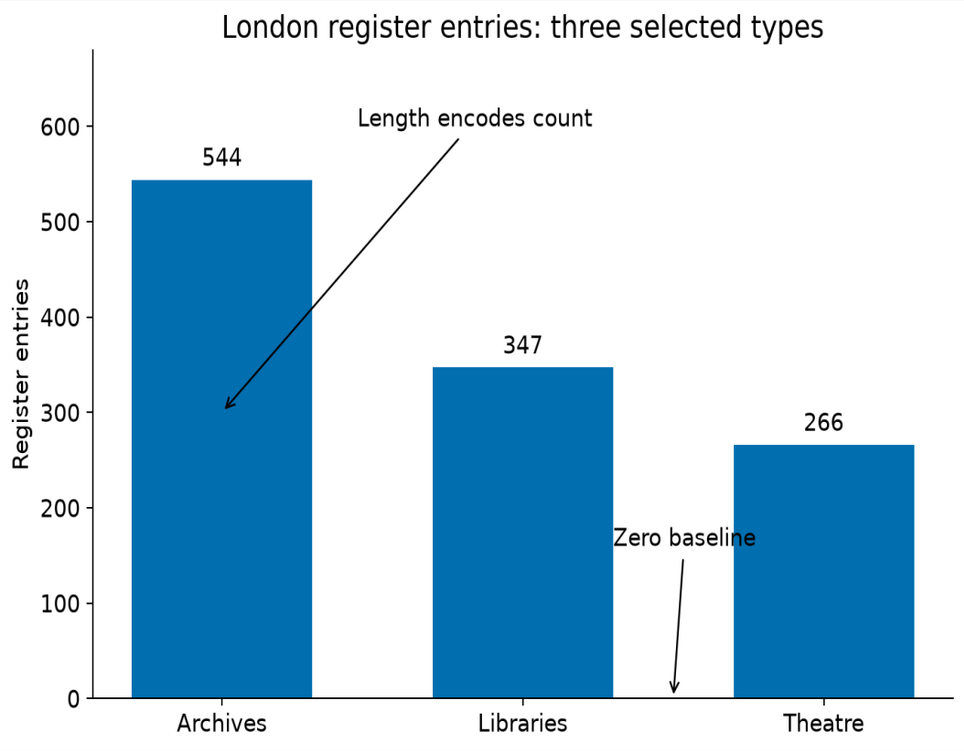
Data and question	London example	Useful chart
Nominal category: counts	venue type	bars, frequency order
Ordinal category: counts	Low → Medium → High	bars, defined order
Numeric variable: distribution	entries per 10k residents	histogram
Two categories: counts or mix	borough × type	grouped/stacked/proportional bars
Comparable values over time	annual counts	line chart
Number by group / two numbers	rate by band / rate versus population	box plot / scatter (later)

A numeric count can have a **distribution** too. Choose by the question as well as the dtype.

Bar plots

1. **Reuse a table:** frequencies, crosstabs or grouped summaries.
2. **Draw readable bars:** labels, order, units and a zero baseline.
3. **Choose the comparison:** counts, rates, totals or composition.
4. **Read the result:** venue records, boroughs and bike hires count different things.

Anatomy of a bar plot



- One separated bar per category
- Length represents a count, proportion or other stated summary
- Zero baseline supports length comparisons
- Category labels identify groups; value axis names measure and units

Our first chart counts **register entries**, not distinct venues.

One plotting pattern

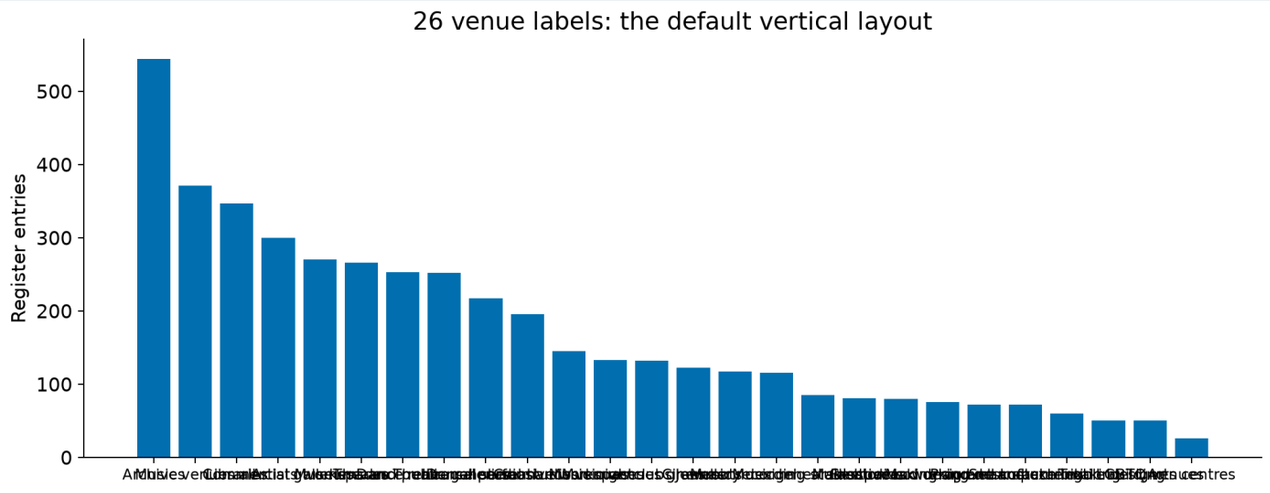
```
import pandas as pd
import matplotlib.pyplot as plt
venues = pd.read_csv(data_path("london_cultural_venues_2023.csv"))
borough_context = pd.read_csv(data_path(
    "london_cultural_infrastructure_2023_borough_context.csv"))
fig, ax = plt.subplots(figsize=(9, 4))
```

`fig` is the whole figure; `ax` is the plotting area. Draw on `ax`, label it, then save `fig`.

The notebooks supply `data_path()` to find local files or the Colab data checkout.

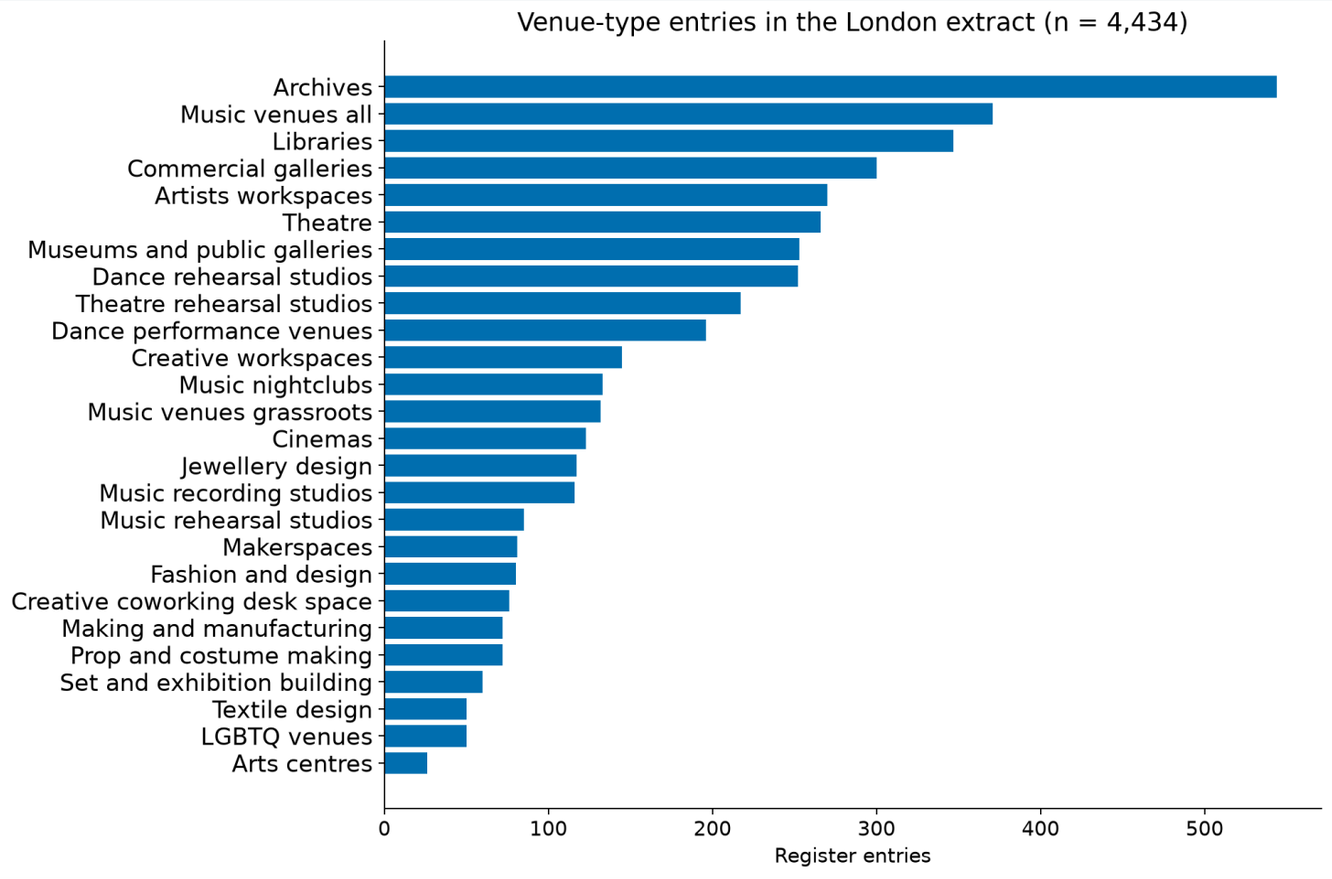
First attempt: 26 long labels

```
counts = venues["venue_type"].value_counts()
fig, ax = plt.subplots(figsize=(11, 4))
ax.bar(counts.index, counts.values)
ax.set_ylabel("Register entries")
```



The counts are correct. The category labels are unreadable.

Horizontal bars fit the labels




```
fig, ax = plt.subplots(
    figsize=(11, 7))
ax.barh(counts.index,
        counts.values)
ax.invert_yaxis()
ax.set_xlabel(
    "Register entries")
```

Archives: 544 entries. The largest register category does not establish the most common kind of distinct cultural place.

Reuse Monday’s ordered bands

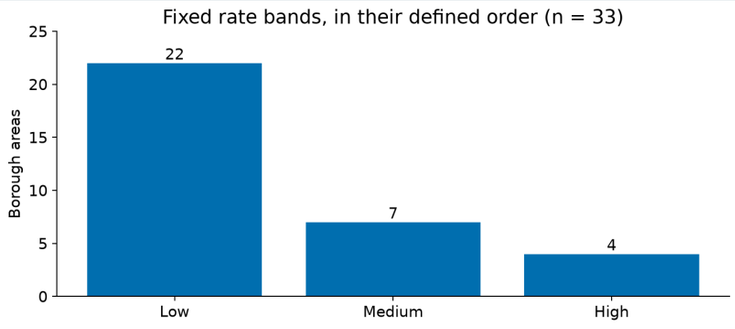
Assign labels with the conditions we already used, then record their order.

```
rate = borough_context["venues_per_10k_residents"]
borough_context["rate_band"] = ""
borough_context.loc[rate < 5, "rate_band"] = "Low"
borough_context.loc[(rate >= 5) & (rate < 15), "rate_band"] = "Medium"
borough_context.loc[rate >= 15, "rate_band"] = "High"
borough_context["rate_band"] = borough_context["rate_band"].astype("category")
borough_context["rate_band"] = borough_context["rate_band"].cat.set_categories(
    ["Low", "Medium", "High"], ordered=True)
```

These are our teaching thresholds. A band records a classification decision, not an official standard of provision.

Ordering is an analytical choice

```
1  bands = borough_context["rate_band"].value_counts(sort=False)
2  fig, ax = plt.subplots(figsize=(8, 3))
3  ax.bar(bands.index, bands.values)
4  ax.set_ylabel("Borough areas")
```



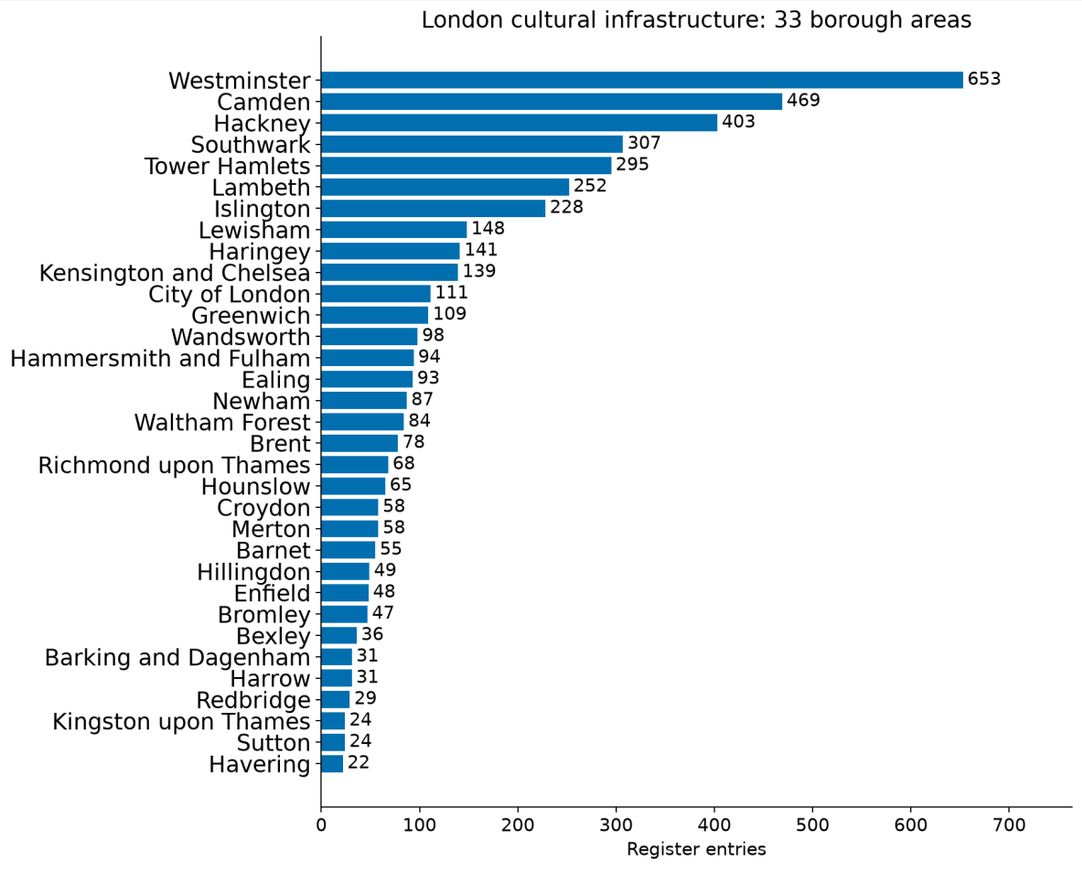
Nominal types can follow frequency. Ordinal bands retain their **defined order**.

Titles, labels, colour and limits

```
ax.set_title("London cultural infrastructure: venue-type entries")
ax.set_xlabel("Register entries")  # horizontal chart
ax.set_xlim(left=0)
ax.tick_params(axis="y", labelsize=11)
fig.tight_layout()
```

- Name the place, measure and relevant date or release.
- Preserve meaningful labels; explain abbreviations.
- Use one colour when colour encodes no extra information.
- Check long labels in the saved file at the intended size.

Borough counts: Westminster leads

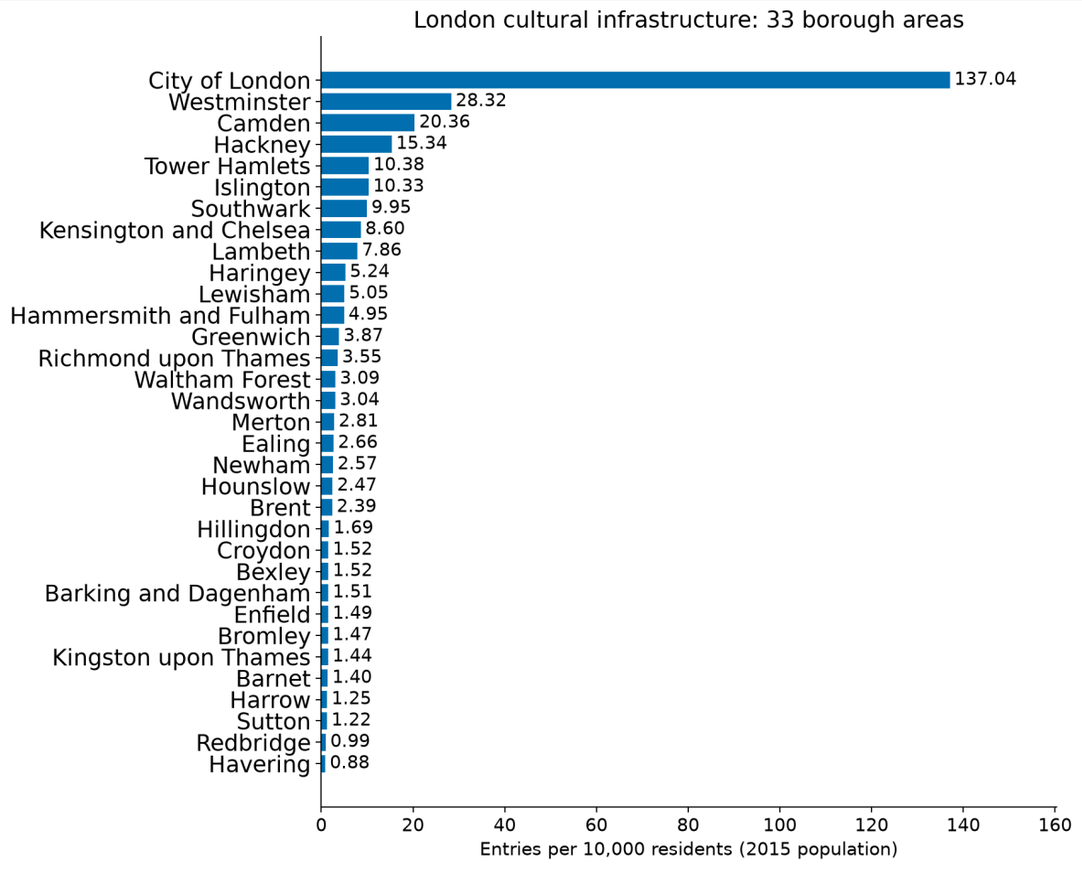


653 Westminster entries, compared with **111 City entries**.

Counts locate more entries, without adjusting for residents or area.

```
ordered = borough_context
ordered = ordered.sort_values(
    "venues", ascending=False)
fig, ax = plt.subplots()
ax.barh(ordered["borough_name"],
        ordered["venues"])
ax.invert_yaxis()
```

Per resident: the City leads



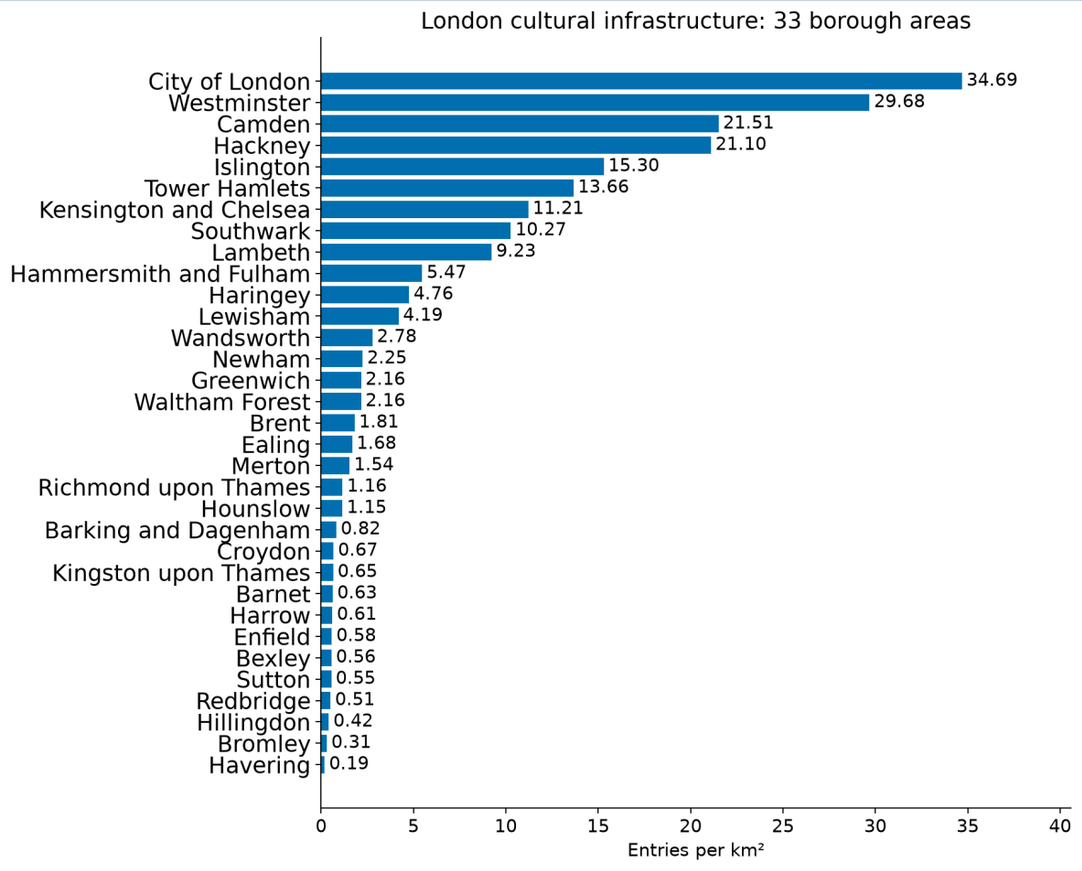
City: **111 ÷ 8,100 × 10,000 = 137.04**.

Westminster: **28.32 entries per 10,000 residents**.

The City serves workers and visitors too. This ratio does not measure individual access.

Population: **2015**. Register: **2023 release**, largely 2022 capture.

Per area: concentration on the ground



City: **111 ÷ 3.2 km² = 34.69 entries/km²**.

Westminster: **653 ÷ 22.0 km² = 29.68 entries/km²**.

Area describes spatial concentration. It does not tell us who can afford, reach or use the places.

Area field: supplied borough profile extract.

Three measures, three questions

Measure	Westminster	City	Question
Register entries	653	111	How many entries here?
Entries per 10,000 residents	28.32	137.04	How many relative to residents?
Entries per km²	29.68	34.69	How concentrated on the ground?

Chart choice includes choosing the quantity to plot. A polished comparison of the wrong measure still answers the wrong question.

Crosstabs: keep the whole denominator

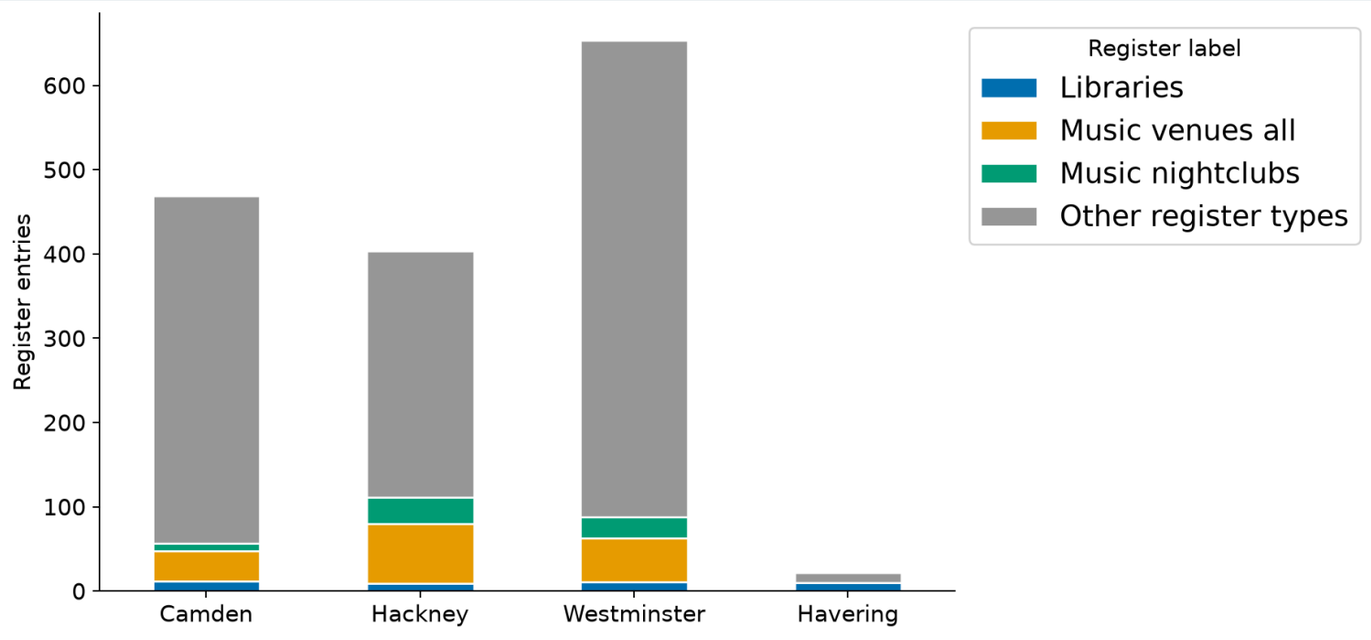
```
boroughs = ["Camden", "Hackney", "Westminster", "Havering"]
types = ["Libraries", "Music venues all", "Music nightclubs"]
ct = pd.crosstab(venues["borough_name"], venues["venue_type"])
mix = ct.loc[boroughs, types].copy()
mix["Other register types"] = ct.loc[boroughs].sum(axis=1) - mix.sum(axis=1)
```

Borough	Libraries	Music all	Nightclubs	Other	Total
Camden	12	36	9	412	469
Hackney	9	71	31	292	403
Westminster	11	52	25	565	653
Havering	10	0	0	12	22

“Other” combines remaining **register labels**. Places can still enter several registers.

Stacked bars: totals and composition

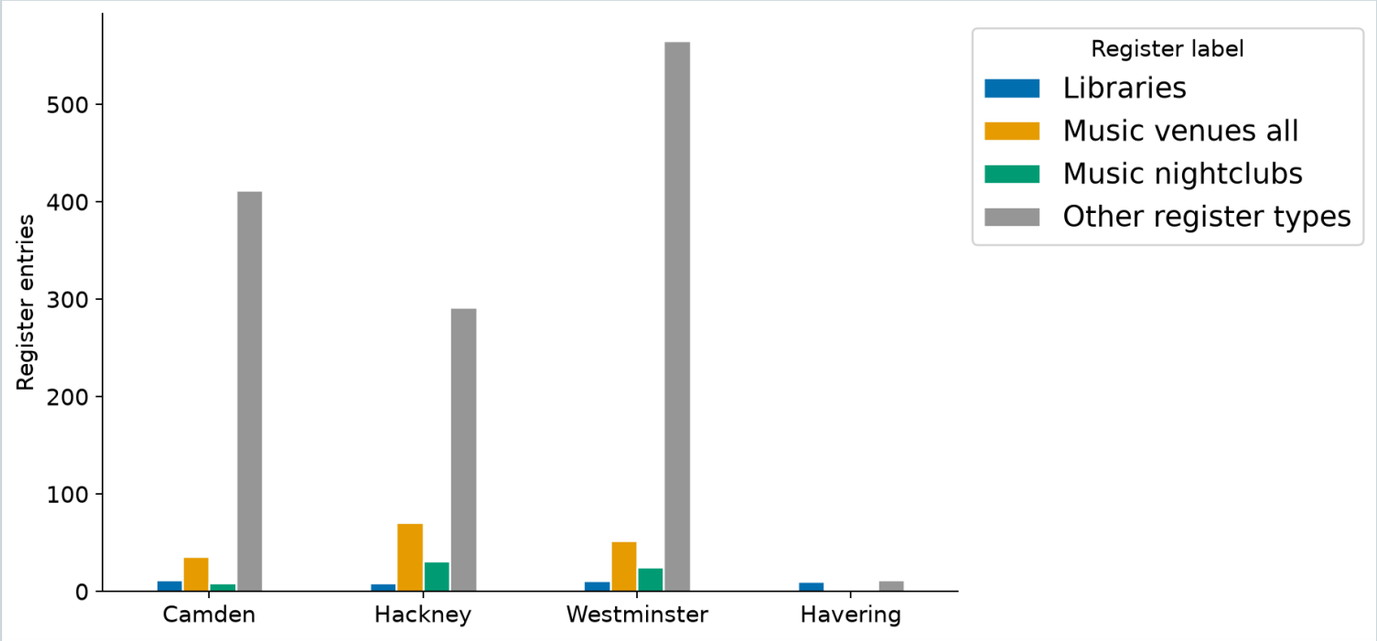
```
palette = ["#0072B2", "#E69F00", "#009E73", "#999999"]
fig, ax = plt.subplots(figsize=(10, 4))
mix.plot.bar(stacked=True, ax=ax, color=palette, rot=0)
ax.set_ylabel("Register entries")
```



Westminster has the largest total. Middle music segments are harder to compare because their baselines differ.

Grouped bars: a common baseline

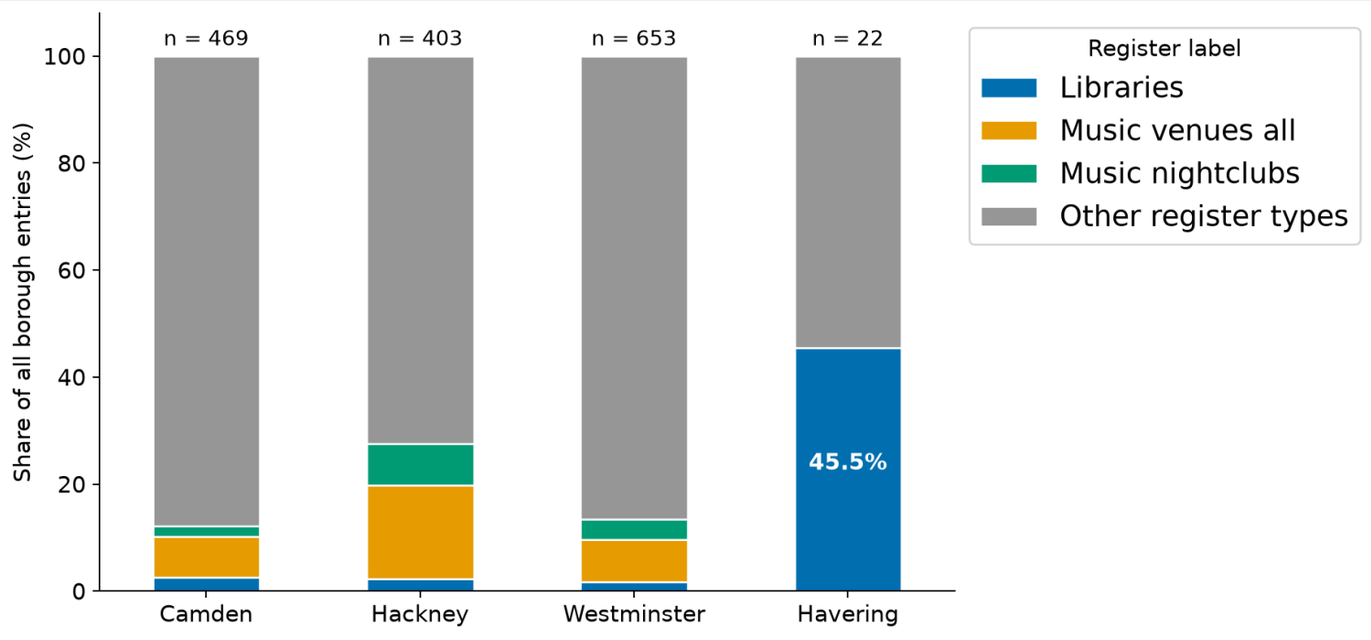
```
fig, ax = plt.subplots(figsize=(10, 4))
mix.plot.bar(ax=ax, color=palette, rot=0)
ax.set_ylabel("Register entries")
```



Hackney's **71 Music venues all entries** exceed Camden's **36**. Each separate bar starts from zero.

Proportional bars: the borough's entry mix

```
1  props = mix.div(mix.sum(axis=1), axis=0) * 100
2  fig, ax = plt.subplots(figsize=(10, 4))
3  props.plot.bar(stacked=True, ax=ax, color=palette, rot=0)
4  ax.set_ylabel("Share of all borough entries (%)")
```



Havering: **10 / 22 = 45.5% Libraries**. Equal heights hide unequal totals, so keep **n** visible.

Which margin? Which subset?

```
row_shares = pd.crosstab(venues["borough_name"],
    venues["venue_type"], normalize="index")
```

Calculation	Havering Libraries	Meaning
Full borough row	10 / 22 = **45.5%**	share of all Havering entries
Only three named types retained	10 / 10 = **100%**	share among selected types

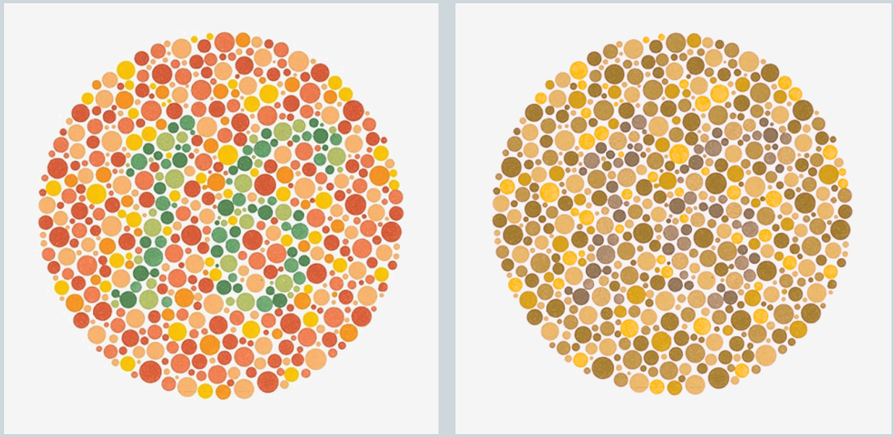
Both calculations can be correct. They concern **different populations of entries**. Normalise the full row before narrowing the display, or retain Other.

Stacked, grouped or proportional?

Layout	Shows well	Limitation	London question
Stacked counts	totals and composition	middle segments lack common baseline	Which total is largest?
Grouped counts	individual values	wider; totals less obvious	Hackney versus Camden music entries?
Proportional stacked	mix despite unequal totals	hides size; middle segments still hard	What is Hackney’s entry mix?

Keep category order and colours consistent. Display counts beside proportions when size matters.

Colour and accessibility



- Use a tested palette, such as Okabe–Ito.
- Keep colours consistent between figures.
- Add labels, borders or line styles; colour alone is insufficient.
- Check contrast and readability at final size.

Our mean and median lines use different **line styles** as well as colours.

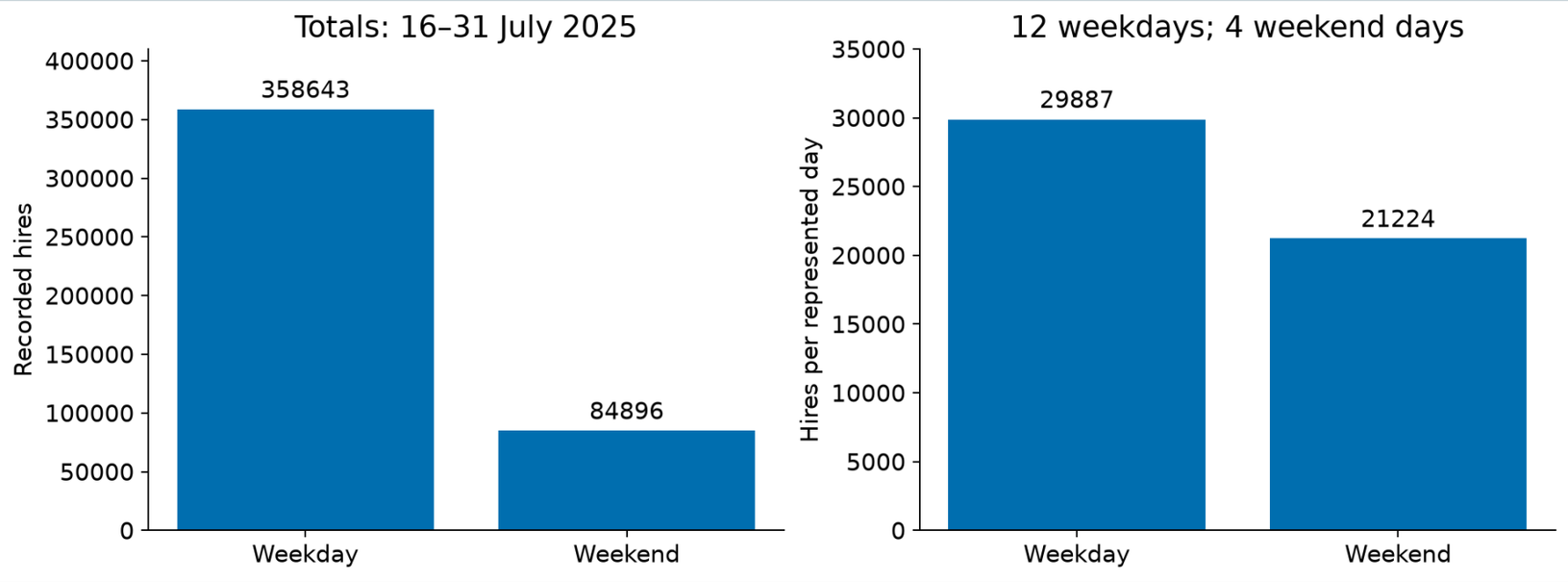
Reuse the weekday and weekend category

Continue with the same **443,539 hires starting on 16–31 July 2025** as Week 5.

```
trips = pd.read_csv(data_path(
    "tfl-bike-hire/bike_trips_16-31Jul2025_with_borough.csv"))
trips["day_type"] = "Weekday"
trips.loc[(trips["weekday"] == "Saturday") |
          (trips["weekday"] == "Sunday"), "day_type"] = "Weekend"
trips["day_type"] = trips["day_type"].astype("category")
day_counts = trips["day_type"].value_counts()
```

One row is one **recorded hire**. The same person may contribute several rows. `day_type` and `duration_min` have **zero missing values** in this extract.

Bike-hire totals and the days represented

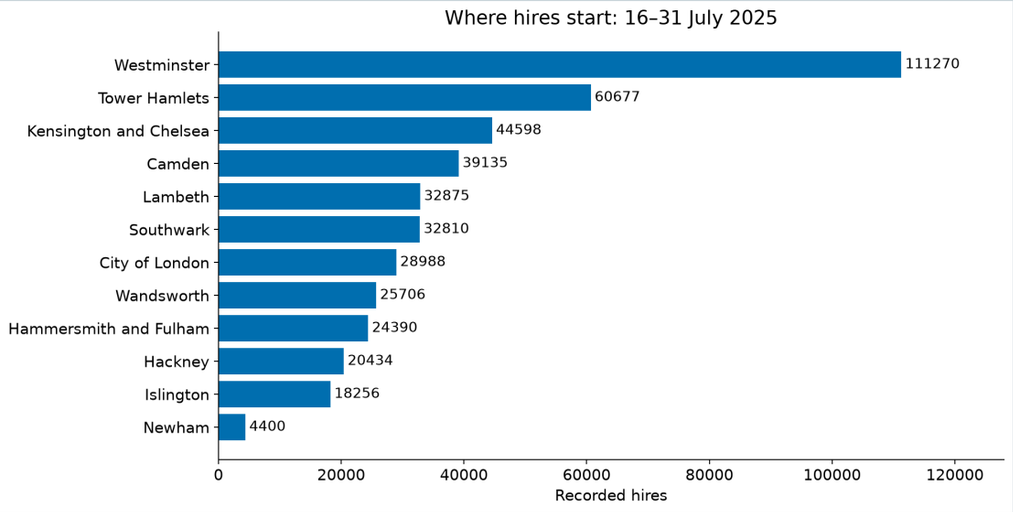


358,643 weekday hires and **84,896 weekend hires** cover **12 weekdays** and **four weekend days**.

Dividing by represented days gives **29,887 versus 21,224 hires per day**. The weekday difference remains, but is much smaller. This short July window does not establish a general annual pattern or trip purpose.

Group counts become a bar plot

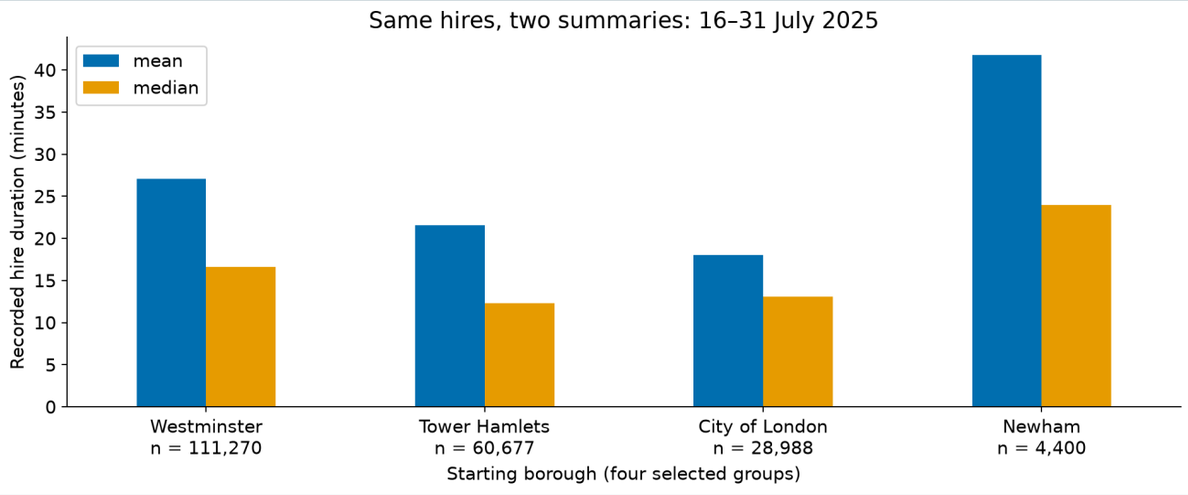
```
hire_counts = trips.groupby("start_borough").size().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(11, 6))
ax.barh(hire_counts.index, hire_counts.values)
ax.invert_yaxis()
ax.set_xlabel("Recorded hires")
```



Westminster: 111,270 starts. These are hires from docking stations in the **12 represented borough areas**, not a ranking of all London cycling demand.

Grouped summaries become grouped bars

```
summary = trips.groupby("start_borough")["duration_min"].agg(
    ["count", "mean", "median"])
shown = summary.loc[["Westminster", "Tower Hamlets", "City of London", "Newham"]]
fig, ax = plt.subplots(figsize=(11, 5))
shown[["mean", "median"]].plot.bar(ax=ax, rot=0)
ax.set_ylabel("Recorded hire duration (minutes)")
```

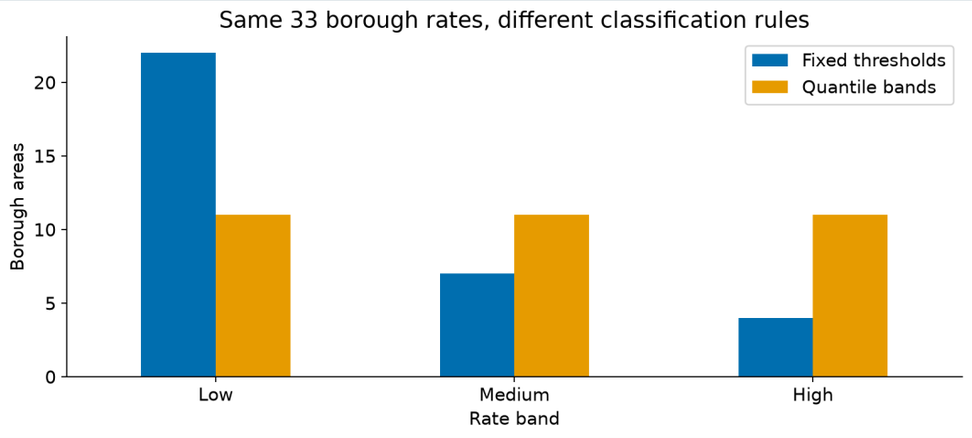


Newham: **mean 41.81, median 24.0 minutes**, from **4,400 hires**. A bar summarises many observations; it does not establish speed, distance or why people hired a bike.

The same rates, two banding rules

Week 5 compared fixed thresholds with approximately equal-sized quantile groups. Draw that same table as grouped bars.

```
borough_context["quantile_band"], edges = pd.qcut(
    rate, q=3, labels=["Low", "Medium", "High"], retbins=True)
band_counts = pd.DataFrame({
    "Fixed thresholds": borough_context["rate_band"].value_counts(sort=False),
    "Quantile bands": borough_context["quantile_band"].value_counts(sort=False)})
fig, ax = plt.subplots(figsize=(9, 4))
band_counts.plot.bar(ax=ax, rot=0)
ax.set_ylabel("Borough areas")
```



Fixed bands: **22 / 7 / 4**. Quantile bands: **11 / 11 / 11**. The rates did not change. Equal group sizes follow from the rule, not evidence of three natural levels of provision.

Distributions and central tendency

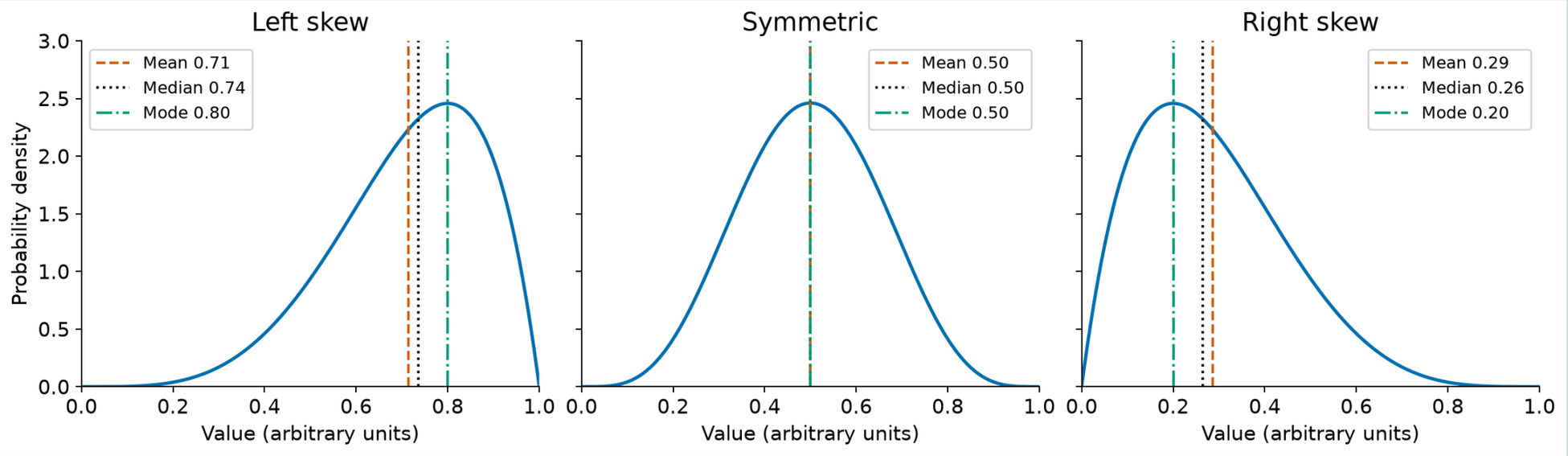
1. **Describe centre and spread:** mean, median, quartiles and range.
2. **Inspect actual values:** historical mortality and recorded hire duration.
3. **Choose bins and a viewing window:** show what is included and omitted.
4. **Investigate unusual cases:** long hires and City of London, before changing data.

Mean, median and mode

Measure	Definition	London use
Mean	sum / number of non-missing observations	equal-weight mean of 33 rates: **9.28**
Median	middle ordered value; average of two middle values when n is even	middle borough rate: **2.81**
Mode	most frequent value or category; ties possible	Archives: **544** entries

Rate units: **entries per 10,000 residents**. A mean of borough rates is not the London-wide rate: borough populations differ.

Distribution shape changes the summary



- A long **left tail** can pull the mean below the median.
- A long **right tail** can pull it above the median.
- In a symmetric unimodal distribution, the centres may coincide.

Income often has a right tail; an easy test may have a left tail. Retirement ages may cluster at institutional thresholds.
Inspect the actual data.

Life expectancy is not a typical age at death

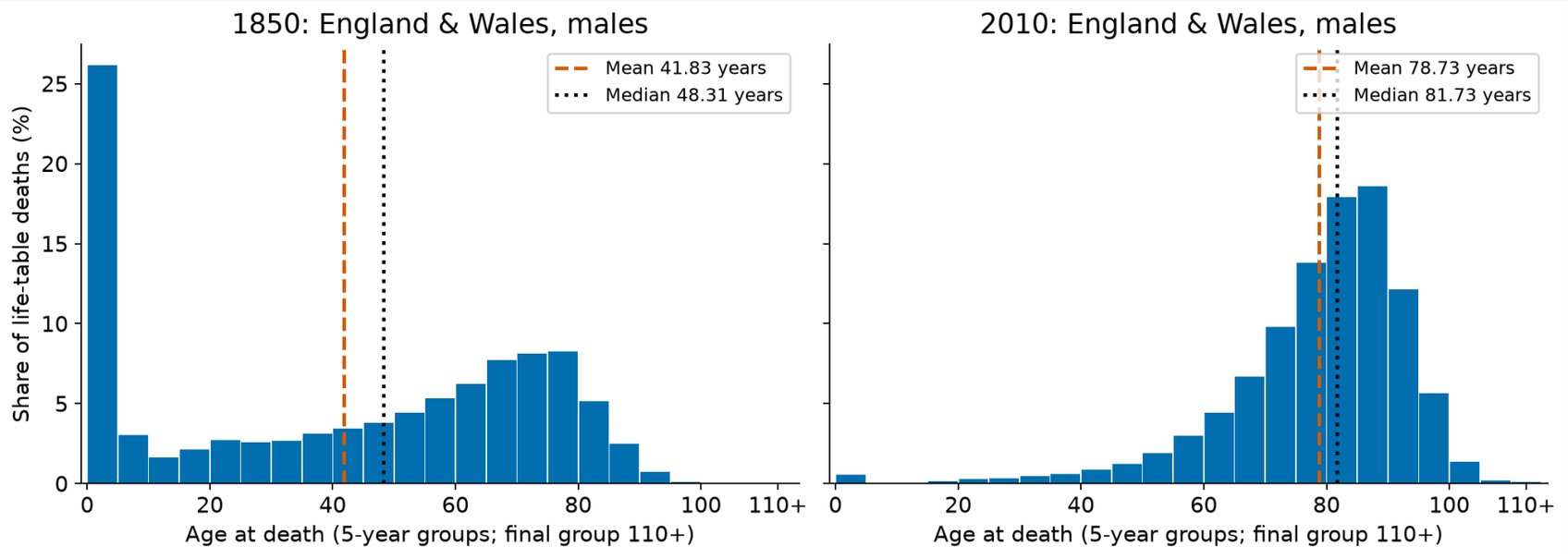


For England, reconstructed life expectancy **at birth** varied around **35–40 years during 1600–1800**.

This does not mean most adults died at 35–40. Infant and child deaths lower the average. Surviving to adulthood changes the conditional expectation; adult mortality mattered too.

Infant mortality: deaths before age one per 1,000 live births. It differs from mortality before age five, ten or fifteen.

Age at death: historical mortality data



1850: 15.5% die before age one under that year's rates; **2010:** 0.465%. Early deaths pull the mean below the median. Adult mortality changed too.

Each period table starts with **100,000 hypothetical births**, exposed throughout life to that year's age-specific risks. Five-year age groups; final group **110+**. These are mortality-based estimates, not a traced birth cohort.

Source: ONS, *Mortality in England and Wales*, Figures 1 and 3 (2012)

The London rate distribution

```
rates = borough_context["venues_per_10k_residents"]
rates.describe()
```

Non-missing n	Mean	Std. dev.	Min	Q1	Median	Q3	Max
33	9.28	23.73	0.88	1.51	2.81	7.86	137.04

Units: **register entries per 10,000 residents**. No rate values are missing.

The mean exceeds three times the median. The distribution helps explain why.

Quartiles, IQR and range

Summary	London result	Meaning
Q1	1.51	lower quartile of ordered rates
Q2 / median	2.81	middle rate
Q3	7.86	upper quartile
IQR = Q3 − Q1	**6.35**	spread of the middle half
Range = max − min	**136.16**	full observed spread

```
q1, q3 = rates.quantile([0.25, 0.75])
iqr = q3 - q1
```

All in **entries per 10,000 residents**. Quantile methods can interpolate; method matters with small n.

Histogram or bar plot?

	Bar plot	Histogram
Groups	named categories	numeric intervals, or bins
Position	often reorderable; preserve ordinal order	follows the numeric scale
Bars	separated	adjoining for contiguous bins
Height today	entry count for a type	area count in a rate interval
Question	Which type has more entries?	Where do rates cluster?

A histogram **classifies numbers into bins**. Like `cut()`, it makes boundaries consequential. It does not preserve borough identities.

Recorded hire durations: read the summary first

```
# One observation is one recorded hire, with duration measured in minutes.
durations = trips["duration_min"]
print(durations.describe().round(2))
```

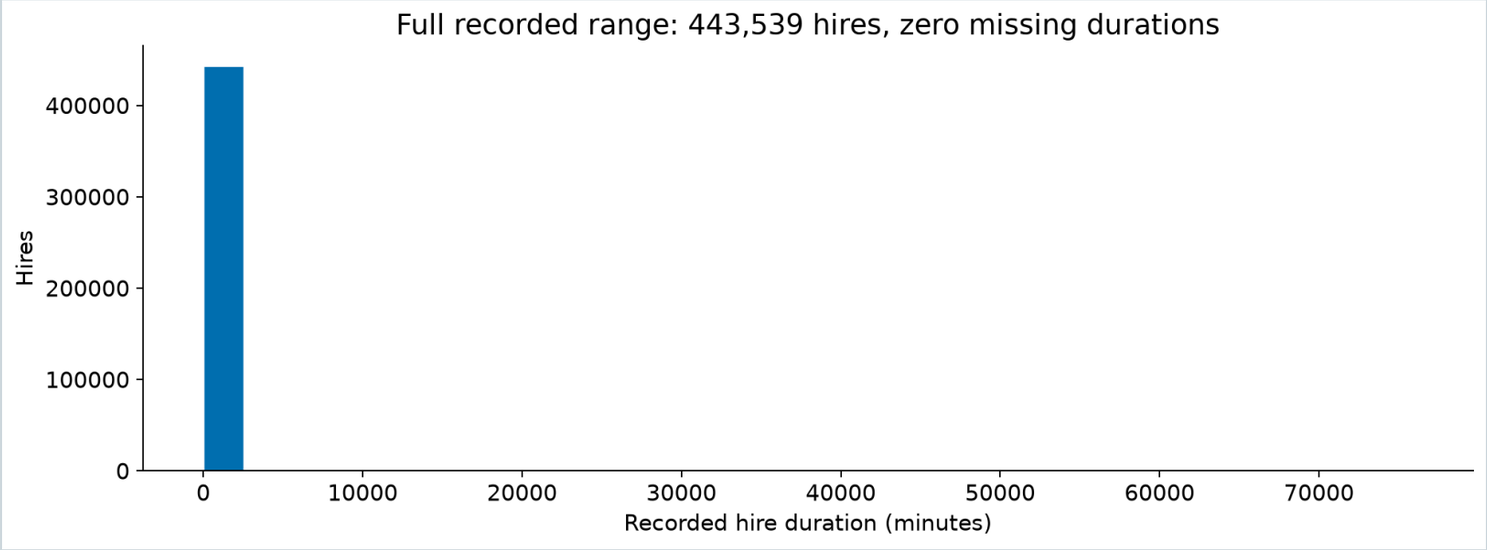
Non-missing n	Mean	Median	Q1	Q3	Minimum	Maximum
443,539	22.97	14.0	8.1	23.0	0	75,882.8

Units: minutes per recorded hire. IQR = 23.0 − 8.1 = **14.9 minutes**. There are **199 zeros** and **269 durations above 1,440 minutes (24 hours)**, retained from Week 5.

The maximum is about **52.7 days**. A summary can alert us to something that needs investigation before we draw the distribution.

All recorded durations: the full range

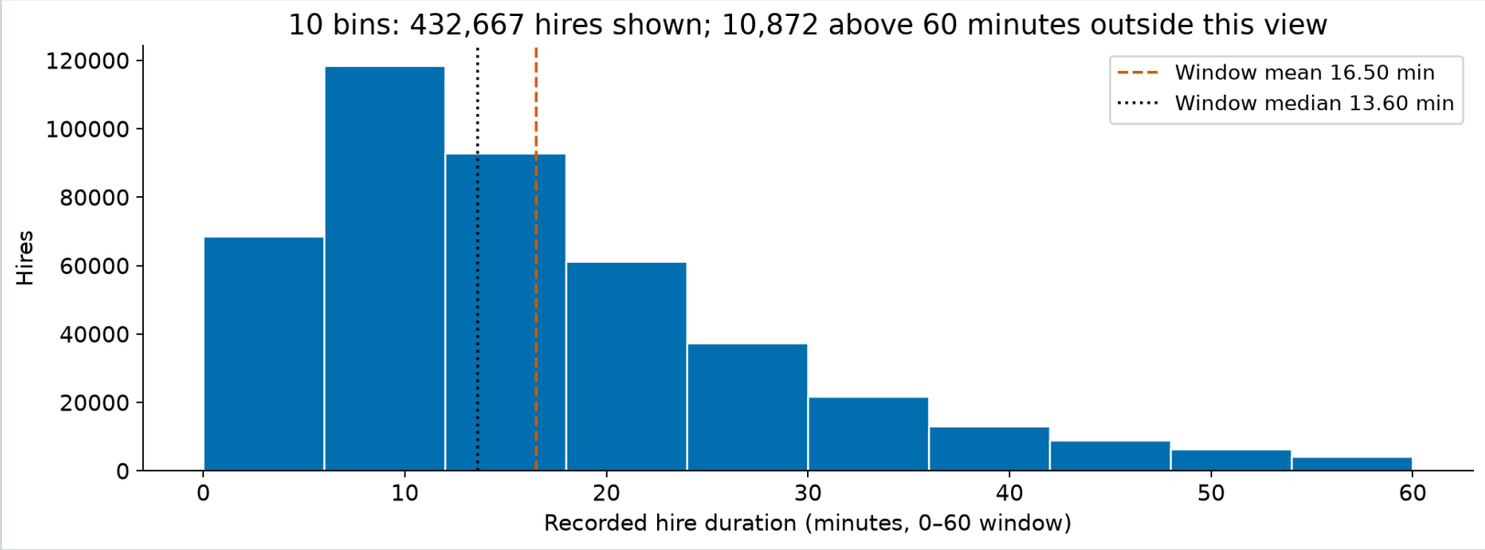
```
fig, ax = plt.subplots(figsize=(11, 4))
ax.hist(durations, bins=30, color="#0072B2", edgecolor="white")
ax.set_xlabel("Recorded hire duration (minutes)")
ax.set_ylabel("Hires")
```



The long range compresses most hires into the first interval. The unusual values are visible as a **scale problem**, while the shorter-duration pattern is hard to read.

A closer view: the first hour

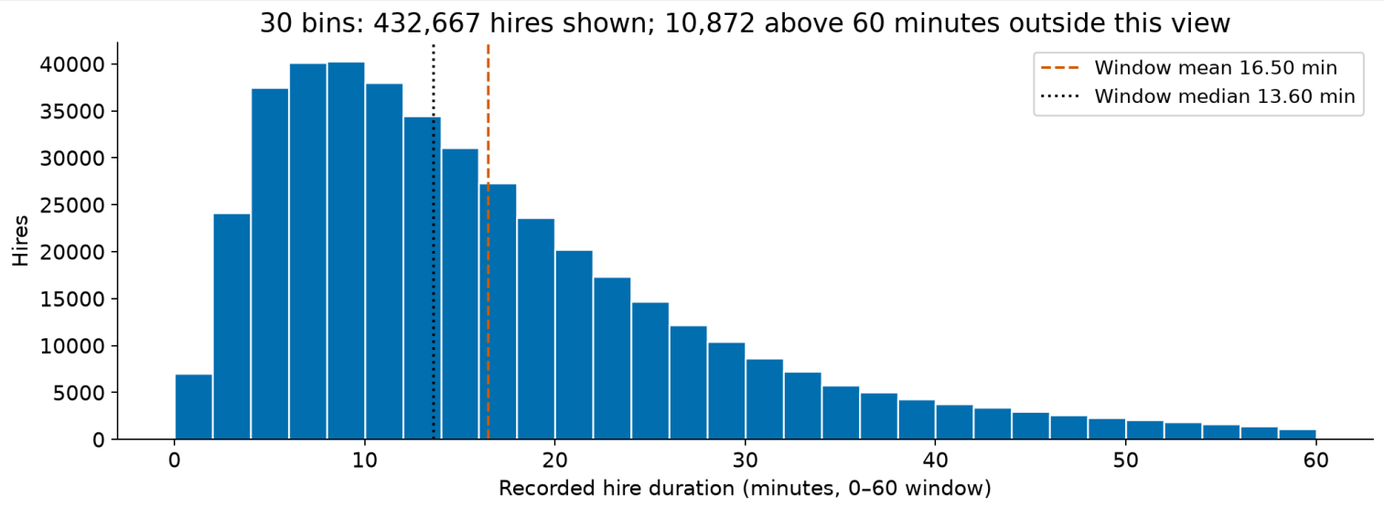
```
within_hour = durations[durations <= 60]
print(len(within_hour), len(durations) - len(within_hour))
fig, ax = plt.subplots(figsize=(11, 4))
ax.hist(within_hour, bins=10, color="#0072B2", edgecolor="white")
```



432,667 hires shown; 10,872 above 60 minutes (2.45%) outside this view. All 199 zeros remain. This viewing choice is not a validated definition of a clean or typical hire.

The viewing window changes the summaries

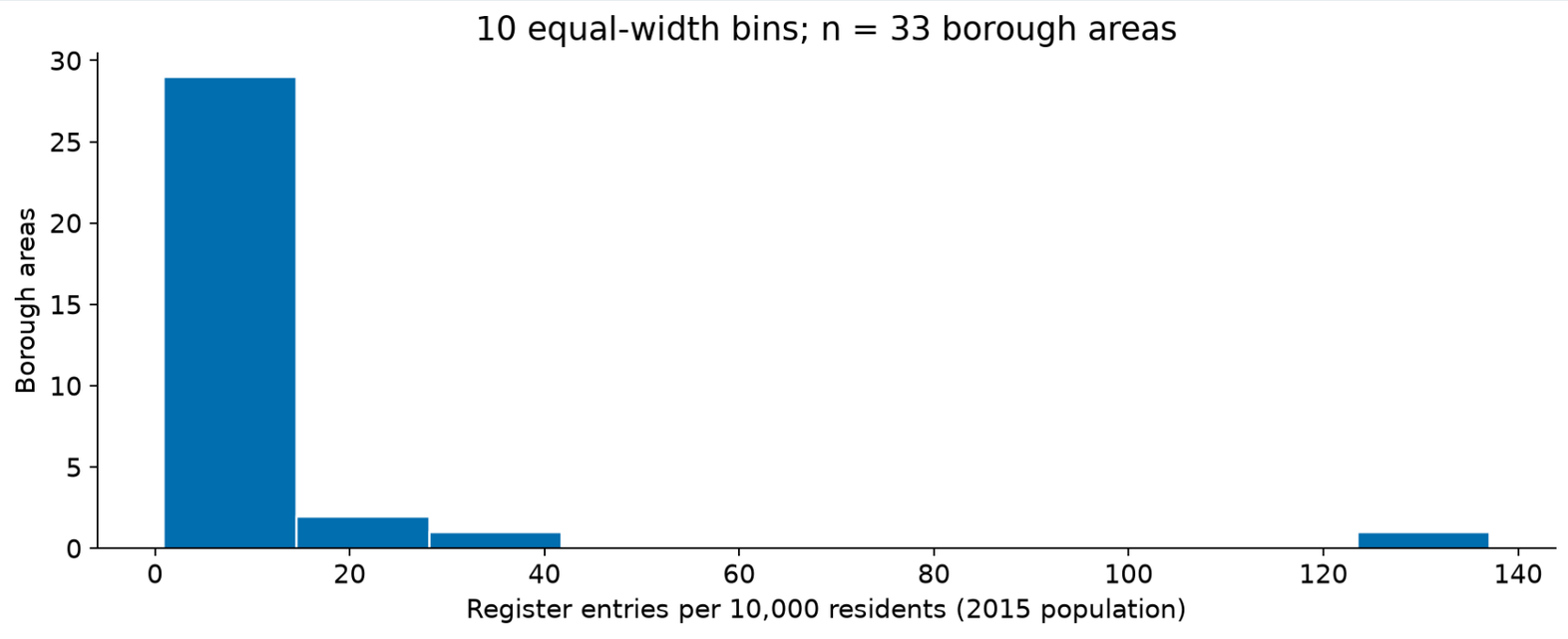
Included hires	n	Mean minutes	Median minutes
All recorded durations	443,539	22.97	14.0
Recorded duration ≤60 minutes	432,667	16.50	13.6



Ten versus thirty bins changes the detail. The **subset rule** changes which hires enter the calculation. Report both decisions and compute each reference line from its stated population.

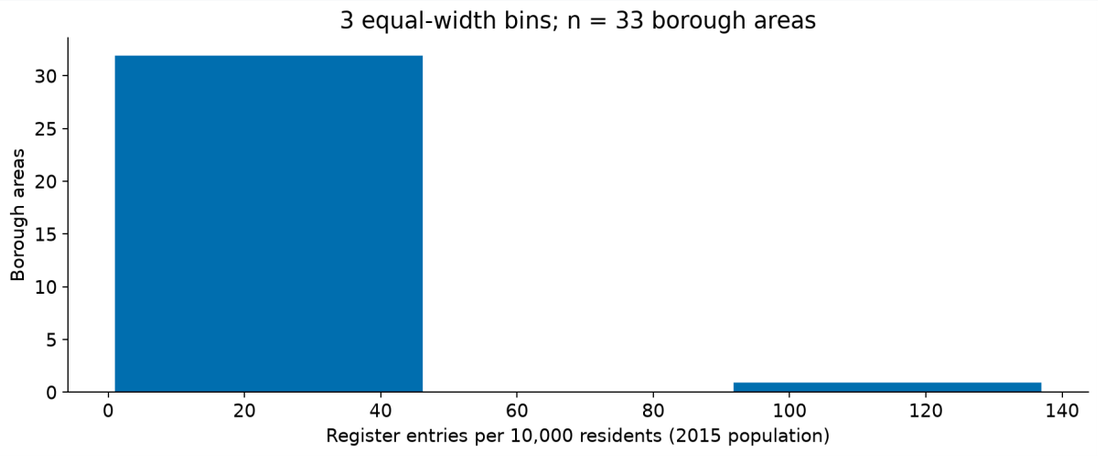
Borough rates: one observation per area

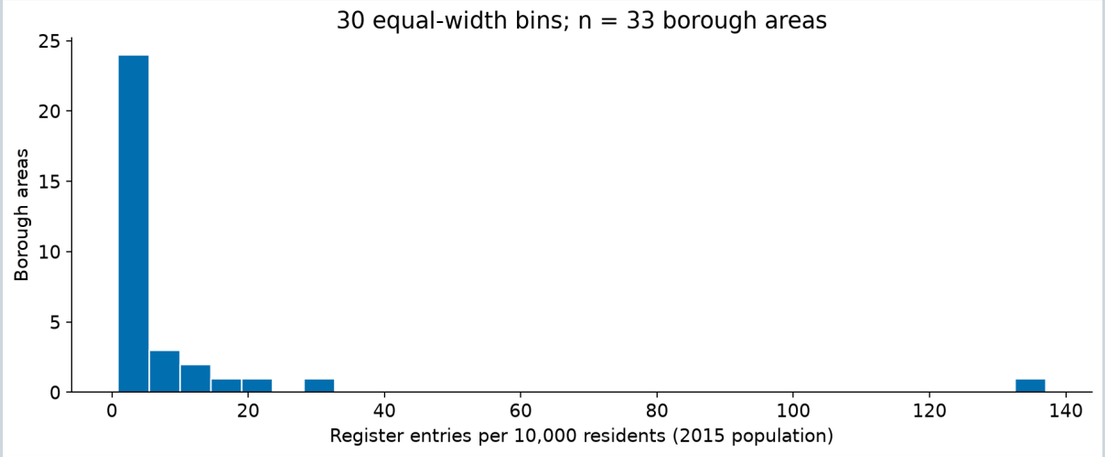
```
fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(rates.dropna(), bins=10, color="#0072B2", edgecolor="white")
ax.set_xlabel("Register entries per 10,000 residents (2015 population)")
ax.set_ylabel("Borough areas")
```



Most areas have low rates; one lies far to the right. Shape becomes visible, but names disappear.

Bins are a classification choice

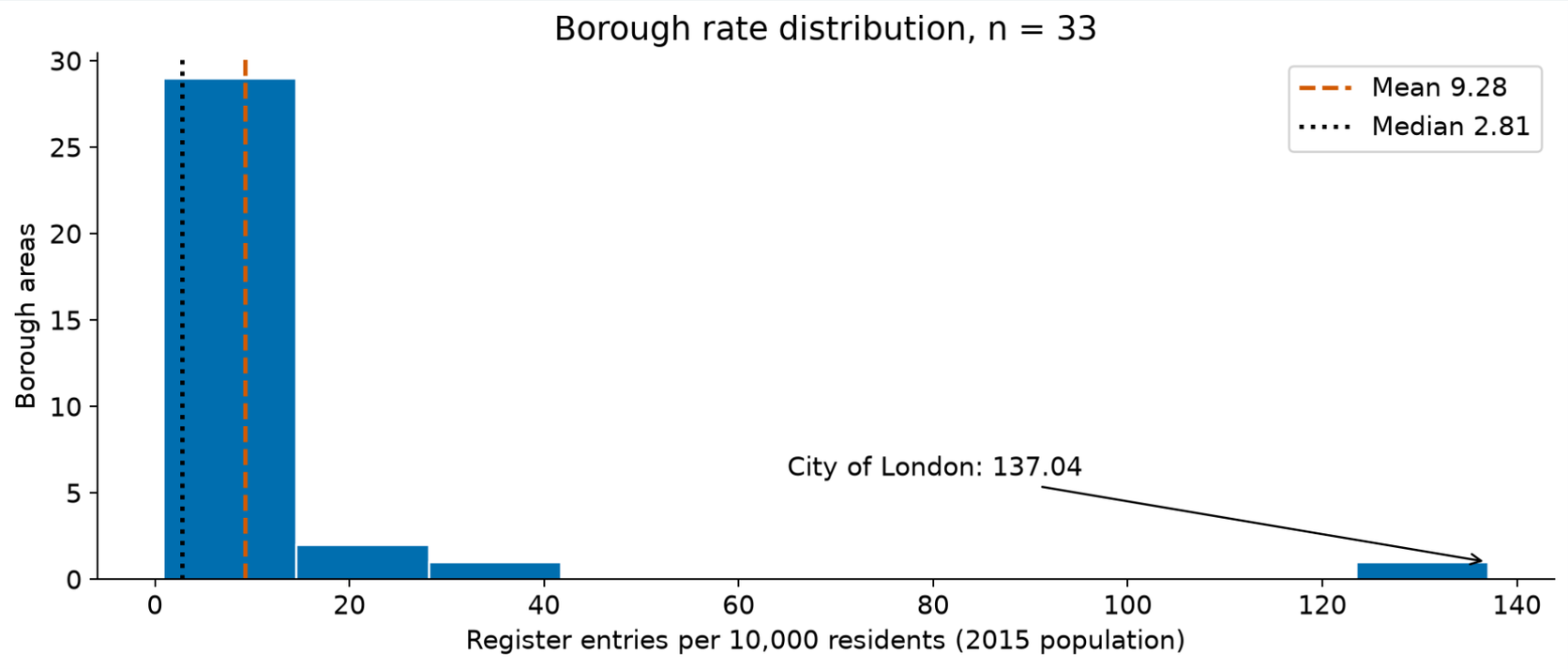




Same **33 values** and numeric range; different boundaries. Three bins hide differences; thirty produce many sparse bins. Neither changes the observations.

With 33 areas, avoid treating every bump as a distinct cultural grouping.

The City: error, outlier or denominator effect?



111 entries / 8,100 residents × 10,000 = 137.04. A small residential denominator makes this a real but extreme value for our definition.

Investigate source row, formula, units, dates and population served. An outlier is **not automatically an error**.

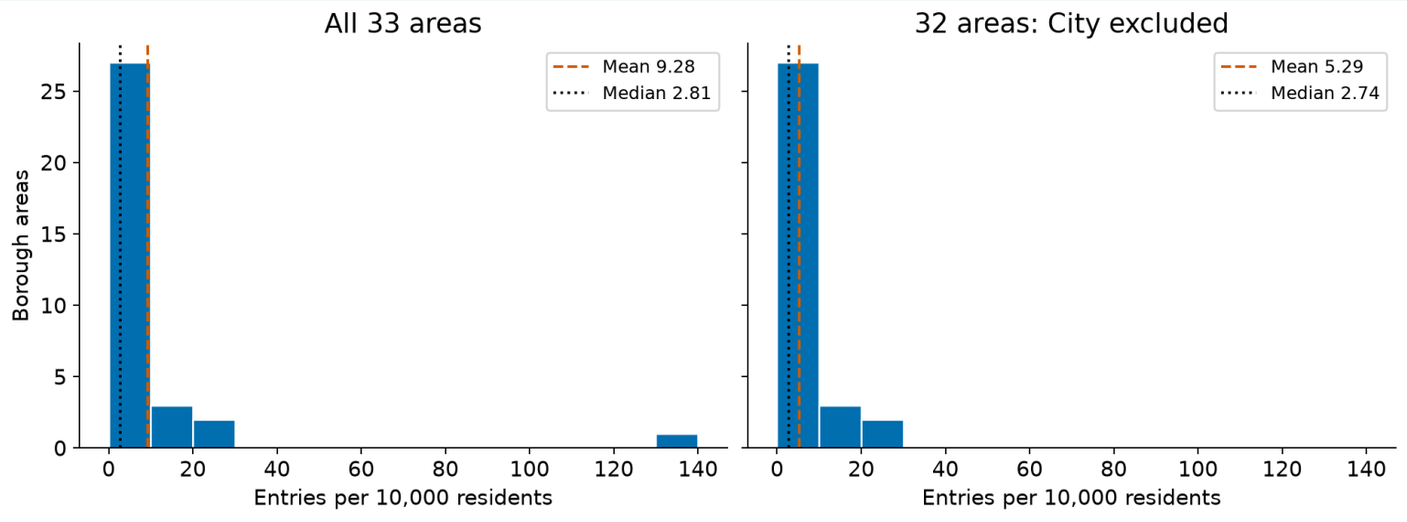
Add mean and median lines

```
ax.axvline(rates.mean(), color="#D55E00", linestyle="--",
           linewidth=2, label=f"Mean {rates.mean():.2f}")
ax.axvline(rates.median(), color="black", linestyle=":",
           linewidth=2, label=f"Median {rates.median():.2f}")
ax.legend()
```

The mean (**9.28**) lies right of the median (**2.81**), pulled towards the high rates. Different line styles preserve the distinction without colour alone.

`axvline()` adds a reference at a numeric x-value. Add lines to the histogram's existing `ax` before saving `fig`.

Sensitivity to one borough



Areas included	n	Mean	Median
All	33	9.28	2.81
City excluded	32	5.29	2.74

A **sensitivity check**, not a corrected dataset. Exclusion changes the population described.

Recompute, rather than hide

```
without_city = borough_context.loc[
    borough_context["borough_name"].ne("City of London"),
    "venues_per_10k_residents"]
print(without_city.agg(["count", "mean", "median"]))
# count 32; mean 5.2878125; median 2.735
```

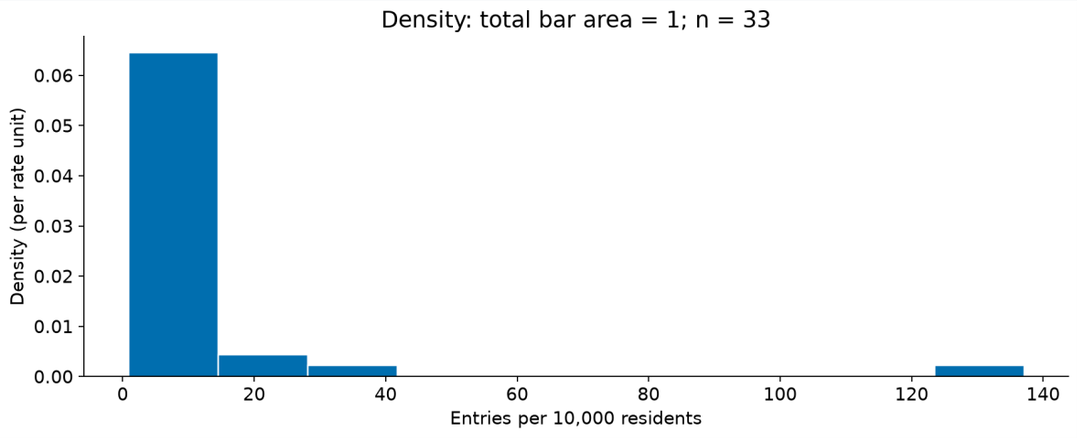
Changing axis limits only hides a plotted value; it does not remove it from `rates.mean()`. A sensitivity calculation requires an explicit subset and a stated reason.

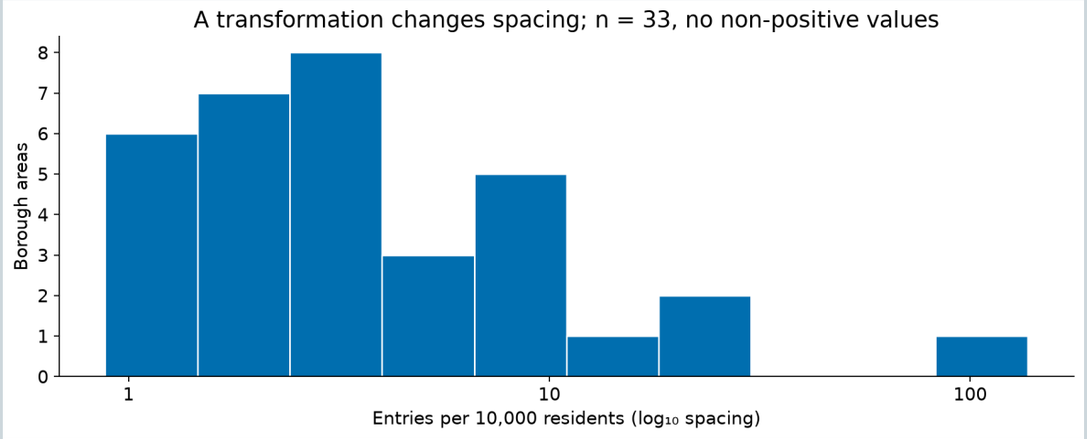
Reading the London histogram

Feature	Observed result	Limit
Centre	median 2.81; mean 9.28	no single rate describes every area
Spread	IQR 6.35; range 136.16	range dominated by City
Shape	long right tail	exact bumps depend on bins
Unusual case	City at 137.04	denominator omits workers/visitors

This is a distribution of **borough-area rates**, with one observation per area. It is not a distribution of residents' cultural experiences.

Counts, density and transformations





`density=True`**:** height = count / (n × bin width). Total **area**, not total height, is one. A smooth density curve is an estimate, not extra data.

Log transformation: equal distances represent ratios. Label the changed scale and check zeros/negatives first.

Histogram best practices

- State observation unit, numeric units, **n** and missing count.
- Try multiple bin settings; report the setting used.
- Hold bin edges and scales fixed when comparing groups.
- Distinguish count, percentage and density axes.
- Investigate unusual values before exclusion.
- Label transformations or restricted plotting ranges.
- Describe observed shape; do not assume normality.

Here: **33 areas, zero missing rates, right skew, City retained**.

Saving and applying figures

1. **Save the named figure:** PNG for raster output, SVG or PDF for vector output.
2. **Open the export:** check labels, units and readability.
3. **Write a caption:** source, observation unit, denominator and limits.
4. **Make a supported claim:** connect the visual pattern to the data it represents.

Save the figure from the notebook

```
from pathlib import Path
Path("figures").mkdir(exist_ok=True)
fig.savefig("figures/borough_rates.png", dpi=200, bbox_inches="tight")
fig.savefig("figures/borough_rates.svg", bbox_inches="tight")
print(Path("figures/borough_rates.png").is_file())
print(Path("figures/borough_rates.svg").is_file())
```

PNG: raster for slides/web. 10 × 4 inches at 200 dpi gives about 2,000 × 800 pixels before tight cropping.

SVG/PDF: scalable vector linework and text. Open exports to check labels. `savefig()` writes the file; no separate device-closing step is required.

What the charts allow us to say

- Archives is the largest **register category**, with 544 entries.
- Westminster leads counts; City leads resident and area rates, with different gaps.
- Havering’s ten library entries form 45.5% of a small total of 22.
- Full recorded hire durations have mean **22.97** and median **14.0 minutes**; a first-hour view changes both the included hires and the summaries.
- Borough resident rates are strongly right-skewed: mean 9.28, median 2.81; City is influential.

These findings depend on **units, definitions, coverage, dates and denominators**. They do not rank cultural value or establish residents’ access.

Resources

- Matplotlib gallery and pandas plotting guide
- Edward Tufte, *The Visual Display of Quantitative Information* (2001)
- Alberto Cairo, *The Truthful Art* (2016)
- ColorBrewer and Data Visualisation Catalogue
- Du Bois visualisation resources
- D'Ignazio and Klein, *Data Feminism*

Live coding and exercises

1. **Reproduce the lecture:** venue bars, borough rates and crosstab charts.
2. **Reuse Week 5 grouping:** turn bike-hire counts and duration summaries into bars.
3. **Inspect distributions:** bins, missingness, extreme values and viewing windows.
4. **Apply and report:** explain a new comparison and save a readable figure.

Exercise 1: bars and denominators

1. Plot all venue-type frequencies horizontally, with a correct count-axis label.
2. Identify the largest category. Why is this not a count of distinct places?
3. Draw borough counts and entries per 10,000 residents in separate figures.
4. Name the leader in each; write a sentence explaining the change.
5. Compare per-km² rates. Which question does each denominator answer?

Include the population year and a limit on what the rates measure.

Exercise 2: which chart answers the question?

1. Make the four-borough crosstab: Libraries, both music labels and Other.
2. Draw stacked, grouped and proportional versions.
3. Choose a chart for **“What is Hackney’s recorded entry mix?”** Explain why.
4. Label totals; explain Havering’s 45.5% with numerator and denominator.
5. Retain only the three named types before normalising. Why does Havering become 100% Libraries?

Why can these entry counts still not be added as distinct places?

Exercise 3: grouped hires and their distribution

1. Reuse `groupby()` on `bike_model`: count hires and calculate mean/median `duration_min`.
2. Draw grouped bars of those two summaries. Keep each group's count beside the chart.
3. Inspect all durations, then draw the lecture's **0–60 minute window** with two bin settings.
4. Report included/excluded counts; retain zeros. Calculate reference lines from the stated subset.
5. Explain why mean and median may order groups differently, and why neither establishes speed or trip purpose.

Use the Week 5 trip file and grouping pattern. The viewing window is an explicit analytical choice, not a validated cleaning rule.

Exercise 4: a distribution and an unusual case

1. Check rate missingness; run `describe()`.
2. Calculate IQR and range with units.
3. Make histograms with two bin settings. What persists; what changes?
4. Add mean and median lines with different styles.
5. Check City’s numerator/denominator; recompute without it and report both n values.
6. Explain why valid arithmetic can still be a limited measure of access.

Exercise 5: save and inspect

1. Save a labelled figure as PNG and SVG; check both files exist.
2. Open both. Check units, legend, clipping and report-size readability.
3. Write a caption naming the observation unit, release, denominator, missing count and one limit.

Own project data: make a useful bar plot and histogram, each with a sentence about what it shows and cannot establish. If no suitable numeric measure exists, explain what additional data are needed rather than plotting an identifier.